

# UT.2. HTML

|                                                           |    |
|-----------------------------------------------------------|----|
| 1. INTRODUCCIÓN A HTML Y XHTML.                           | 3  |
| 1.1. HISTORIA DE HTML Y XHTML                             | 3  |
| 1.1.1. VERSIONES DE HTML Y XHTML                          | 4  |
| 2. HERRAMIENTAS                                           | 6  |
| 2.1. EDITOR DE TEXTO                                      | 6  |
| 2.2. NAVEGADOR                                            | 6  |
| 3. SINTAXIS HTML                                          | 7  |
| 3.1. ETIQUETAS HTML                                       | 7  |
| 3.2. COMENTARIOS                                          | 9  |
| 3.3. ESTRUCTURA DE UNA PÁGINA HTML                        | 9  |
| 3.4. DECLARACIÓN DEL TIPO DE DOCUMENTO                    | 10 |
| 3.5. ETIQUETA HTML                                        | 10 |
| 3.6. ETIQUETA HEAD                                        | 11 |
| 3.6.1. TITLE                                              | 11 |
| 3.6.2. META                                               | 11 |
| 3.6.3 LINK                                                | 15 |
| 3.6.4. SCRIPT                                             | 15 |
| 3.6.5. STYLE                                              | 16 |
| 3.7. CONTENIDO DEL CUERPO. ETIQUETA <body>                | 17 |
| 3.7.1. MANEJO DEL TEXTO                                   | 20 |
| 3.7.1.1. ENCABEZADOS                                      | 20 |
| 3.7.1.2. PÁRRAFOS                                         | 22 |
| 3.7.1.3. SALTOS DE LÍNEA                                  | 23 |
| 3.7.1.4. LÍNEA HORIZONTAL                                 | 24 |
| 3.7.1.5. ESTILOS DE FUENTE                                | 25 |
| 3.7.1.6. TEXTO PREFORMATEADO                              | 31 |
| 3.7.2. LISTAS                                             | 33 |
| 3.7.2.1. LISTAS DE ELEMENTOS ORDENADAS                    | 33 |
| 3.7.2.2. LISTAS NO ORDENADAS                              | 34 |
| 3.7.2.3. LISTAS DE DEFINICIONES                           | 36 |
| 3.7.3. ENLACES                                            | 38 |
| 3.7.3.1. ENLACES A OTROS SITIOS                           | 38 |
| 3.7.3.2. ENLACES A OTRAS PÁGINAS DEL MISMO SITIO          | 39 |
| 3.7.3.3. ENLACES A UN SITIO ESPECÍFICO DE LA MISMA PÁGINA | 42 |
| 3.7.3.4. ENLACES A UN SITIO ESPECÍFICO DE OTRA PÁGINA     | 43 |
| 3.7.3.5. PROTOCOLOS ESPECIALES EN ENLACES                 | 43 |
| 3.7.4. IMÁGENES                                           | 44 |
| 3.7.4.1. FORMATOS DE IMAGEN                               | 46 |
| 3.7.4.2. ETIQUETAS <figure> Y <figcaption>                | 47 |
| 3.7.5. TABLAS                                             | 48 |
| 3.7.5.1. AGRUPANDO CELDAS                                 | 49 |
| 3.7.5.2. ESTRUCTURANDO LA TABLA                           | 50 |

|                                                                |    |
|----------------------------------------------------------------|----|
| 3.7.6. FORMULARIOS                                             | 51 |
| 3.7.6.1 MÉTODO DE ENVÍO DEL FORMULARIO                         | 54 |
| 3.7.6.2 <input>. CAMPOS DE ENTRADA DE DATOS                    | 54 |
| 3.7.6.3 <textarea>. CAMPOS DE ENTRADA DE TEXTO MULTILÍNEA      | 65 |
| 3.7.6.4 <label>. ETIQUETANDO LOS CONTROLES DEL FORMULARIO      | 66 |
| 3.7.6.5 <select>. CAJAS DE LISTAS DESPLEGABLES                 | 67 |
| 3.7.6.5.1 <option> y <optgroup>. AÑADIENDO OPCIONES A LA LISTA | 67 |
| 3.7.6.6 <datalist>. LISTA DE SUGERENCIAS                       | 69 |
| 3.7.6.7 <fieldset> Y <legend>. ESTRUCTURANDO EL FORMULARIO     | 70 |
| 3.7.6.8 ALGUNOS ATRIBUTOS COMUNES A ELEMENTOS DE <form>        | 70 |

# 1. INTRODUCCIÓN A HTML Y XHTML.

**HTML** es el principal lenguaje informático con el que se escriben las páginas web. Las páginas web son vistas por el usuario mediante unos programas llamados navegadores o "**browsers**".

Por tanto, HTML es uno de los **lenguajes** que usan los navegadores para mostrar las páginas webs al usuario. Permite insertar textos, sonidos e imágenes y combinarlos a nuestro gusto.

Además, permite introducir referencias a otras páginas Web por medio de los enlaces hipertexto.

## 1.1. HISTORIA DE HTML Y XHTML

Hablar de la historia de HTML sin mencionar el nombre Tim Berners Lee es imposible. A principios de la década de los 90, el físico Tim Berners Lee, investigador de la Organización Europea para la Investigación Nuclear (CERN), realizó una publicación titulada «HTML Tags», conocida como el origen de HTML. Además, unos años después, fundó la W3C, que es uno de los principales agentes en la historia de HTML, como verás más adelante.



Esta publicación respondía a la **necesidad de crear un sistema de hipertexto que les permitiera a los científicos e investigadores del CERN intercambiar información de forma rápida y fácil de referenciar**. Ten presente que la investigación nuclear reúne a científicos de diversas disciplinas, por lo que optimizar el intercambio de información resultaba urgente. Es decir, **HTML (*HyperText Markup Language*) surgió con el objetivo de agilizarles sustancialmente el trabajo y, por supuesto, lo logró**.

### Pasado

Ya hemos visto en el tema anterior que los **lenguajes de marcas** son bastante antiguos, y sus inicios se pueden situar realmente con el lenguaje SGML (Standard Generalized

Markup Language). Este lenguaje en realidad nunca llegó al gran público, aunque sigue existiendo y usándose en entornos muy específicos.

No es hasta la llegada de **Internet** que los lenguajes de marcas toman la relevancia y popularidad que tienen hoy en día.

## Presente

El lenguaje de marcas más usado y conocido hoy es **HTML (HyperText Markup Language)**. Ha ido cambiando mucho a lo largo del tiempo. La versión más actual es la conocida como HTML5.

El fundamento de Internet y todas las tecnologías asociadas se basa en estándares abiertos e independientes de la tecnología. El organismo que regula estos estándares sin ninguna contrapartida a cambio es el [World Wide Web Consortium \(W3C\)](http://www.w3.org/). Todos sus estándares (que ellos llaman Technical Reports) pueden descargarse gratuitamente en su web.

## Futuro

El futuro sigue estando en sistemas abiertos, que facilitan la competencia y benefician al usuario. La web ha tenido un crecimiento exponencial debido a las plataformas móviles, que han multiplicado los accesos a las páginas web y que también han dificultado el diseño web.

Hablaremos más de ello en el tema sobre CSS Cascade Style Sheet.

### 1.1.1. VERSIONES DE HTML Y XHTML

Aunque hay una versión 1.0 de HTML de 1991, es la versión 2.0 de 1995 la que se considera la primera propuesta oficial de estandarización de HTML.

Dos años después, **en 1997, se generaron dos nuevas versiones**: a principios de año se publicó la **versión HTML 3.2** y, a finales del mismo año, se publicó la **actualización HTML 4.0**. Como puedes ver, los cambios fueron a toda velocidad y los desarrolladores web de la época estuvieron algo ocupados adaptándose a cada modificación. Esto por no hablar de las pugnas entre los fabricantes de navegadores que realizaban actualizaciones muy personalizadas, hasta el punto de que una página web podía ser incompatible de un sitio a otro.

Posteriormente, a finales del año 1999, se publicó una nueva especificación oficial: la **versión HTML 4.01**, que marcó otro hito para la evolución de HTML.

Por su parte en el año 2000 el W3C publica **XHTML 1.0**: es una reformulación de HTML 4.0 (1998) en XML. El objetivo era **mejorar** las deficiencias que tenía **HTML** en aquel momento, incorporando las ventajas que aportaba un nuevo lenguaje recién creado, el **XML**.

W3C siguió trabajando por este camino en un nuevo lenguaje XHTML 2.0, pero que ya no era compatible ni con el HTML, ni siquiera con XHTML 1.0.

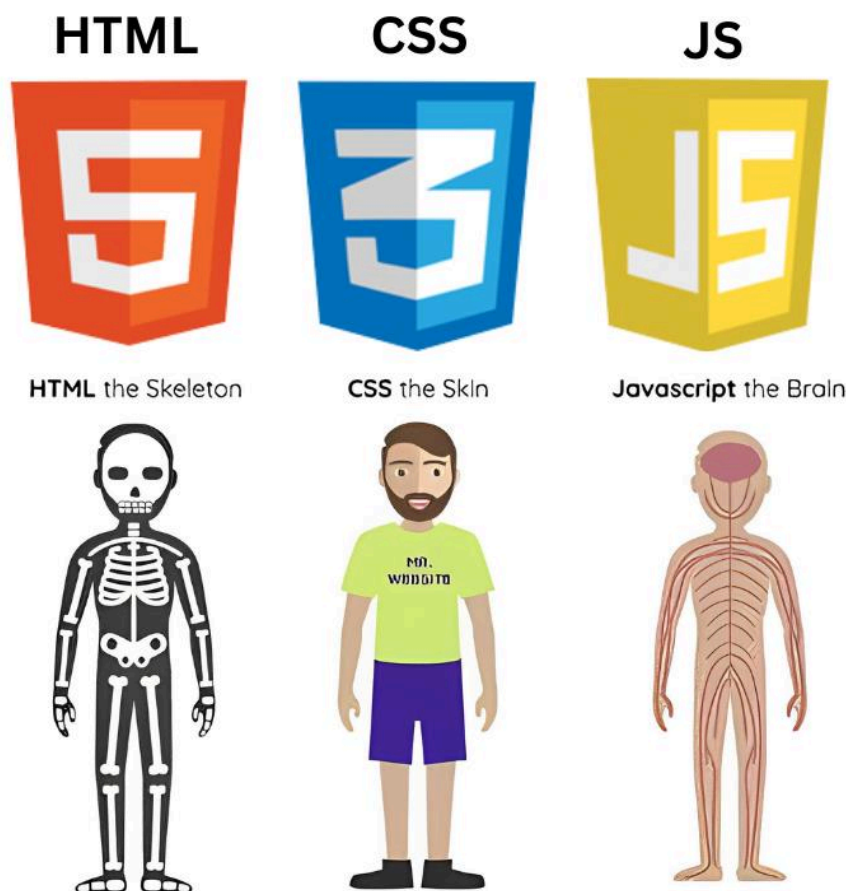
Desde el año 2014, **HTML 5 es estándar** y, aunque siguen surgiendo nuevas modificaciones y actualizaciones, los cambios son mínimos y fáciles de asimilar.

- [HTML 3.2](#), W3C Recommendation, 14 January 1997, Dave Raggett, Author
- [HTML 4.01](#), W3C Recommendation, 24 December 1999, Dave Raggett, Arnaud Le Hors, Ian Jacobs, Editors
- [HTML5](#), W3C Recommendation, 28 October 2014.

Para más información:

- HTML: <http://www.w3.org/community/webed/wiki/HTML/Specifications>
- HTML5:
  - Recomendación [HTML5](#) publicado en Octubre del 2014.
  - [http://www.w3.org/html/wiki/FAQs#When\\_can\\_I\\_use\\_HTML5.3F](http://www.w3.org/html/wiki/FAQs#When_can_I_use_HTML5.3F)
  - <http://es.wikipedia.org/wiki/HTML5>

Existen otros lenguajes que se introdujeron para superar algunos inconvenientes de HTML, ya que este lenguaje se queda corto para organizar, optimizar y automatizar el funcionamiento de las webs. Ejemplos de estas son **CSS y JavaScript**.



## 2. HERRAMIENTAS

En este apartado vamos a ver lo que necesitaremos para seguir este manual y poder realizar los ejemplos que pongamos.

### 2.1. EDITOR DE TEXTO

Los textos escritos de código fuente en los lenguajes HTML y CSS debemos guardarlos en archivos, por lo que necesitamos un **editor de textos plano**.

Cualquier editor de texto sin formato vale aunque, eso sí, los ficheros deben tener extensión html. Se pueden encontrar documentos html con extensión htm, estos ficheros son iguales y no hay diferencia entre ellos. La extensión htm existe porque anteriormente el nombre de las extensiones estaba limitado a 3 caracteres.

En principio cualquier editor de texto es válido, por lo que puede incluso servirnos el bloc de notas de Windows.

Existen editores de html más avanzados que, por ejemplo, codifican las etiquetas con algún sistema de colores, siendo uno de los más utilizados hoy en día es **Visual Studio Code**.

Otros editores: Sublime Text, Netbeans, JetBrains Webstorm, Kompozer, Aptana Studio, CoffeeCup Free Editor, Brackets, TextWrangler y Notepad++.

Nosotros utilizaremos Visual Studio Code con las siguientes consideraciones:

- La única extensión permitida en los exámenes será Live Server.
- Pueden utilizarse abreviaturas Emmet pero, ojo, no todos los entornos de desarrollo las incorporan. En el futuro puedes encontrarte en un entorno que nos las ofrece, por lo que es fundamental que aprendas las sintaxis HTML estándar sin atajos. Puedes consultarlas en [este enlace](#).
- Cualquier uso de extensiones o ayudas que no sean éstas (incluida cualquier tipo de IA) está **completamente prohibido en los exámenes** y supondrá motivo tener que abandonarlo con calificación cero.

### 2.2. NAVEGADOR

El **navegador** es el software que **interpreta el lenguaje HTML**. Existen muchos navegadores, y existen diferencias entre unos navegadores y otros a la hora de interpretar determinado código no estándar HTML, por lo que lo mejor es ajustarse al estándar y no usar etiquetas que hacen cosas llamativas pero que sólo valen para un determinado navegador. Una página web que usa HTML estándar es accesible desde cualquier navegador.

Nuestro objetivo debe ser que cualquier usuario pueda ver bien (tal y como nosotros queremos) nuestra página web. Es absurdo ver mensajes que digan “Página optimizada para...” y ahí un navegador o resolución de pantalla (¿vas a tener que cambiar la resolución o el navegador sólo para ver una página?).

Como es obvio, necesitamos al menos un navegador para ver los resultados de nuestro trabajo, aunque en realidad es conveniente tener instalados varios navegadores. No todos los navegadores funcionan de la misma manera ni muestran las páginas igual. Puede haber pequeñas (o grandes) diferencias entre uno y otro. Debemos **contrastar** los resultados en varios de ellos para garantizar el correcto desarrollo de nuestros programas. Necesitamos evidentemente tener instalados los navegadores más usados por los internautas y que abarquen prácticamente la totalidad de cuota de mercado, para asegurarnos que nuestra página la verán igual (o de forma muy parecida) la gran mayoría de los internautas.

## 3. SINTAXIS HTML

**HTML** es un lenguaje de marcas de tipo **semántico**, por lo que se basa en el hecho de introducir etiquetas dentro de nuestros ficheros para dar significado a ciertas partes del texto, pero, ¿Qué es una etiqueta?

### 3.1. ETIQUETAS HTML

Las **etiquetas** son los textos que delimitan los distintos **elementos** que componen un documento. Los elementos de HTML suelen estar formados por una **marca de inicio** `< >`, un **contenido** y una **marca de final** `</ >`. Si un elemento no tiene contenido se llama **elemento vacío** y no tiene etiqueta de cierre, como por ejemplo `<br>`.

Por tanto, un **elemento HTML** estará constituido por una **etiqueta de inicio**, algún **contenido** y una **etiqueta final**.

`<h1>`Mi primera cabecera`</h1>`

**Etiquetas de inicio de elemento.** Delimitadas por los caracteres “<” y “>”.

`<identificador>`

Ej. `<head>`

**Etiquetas de final de elemento.** Delimitadas por los caracteres “</” y “>”.

`</identificador>`

Ej. </head>

Muchas etiquetas admiten atributos, los cuales modifican el funcionamiento de esta.

Sintaxis:

<identificador [atributos="valor"]>

- Todos los elemento HTML pueden incluir atributos
- Los atributos aportan información adicional a los elementos
- Siempre aparecen en la etiqueta de apertura del elemento
- Habitualmente aparecen en pares nombre/valor en la forma: name="value"

Ejemplo de elemento HTML:

```
<a href="https://www.mozilla.org/es-AR/about/manifesto/" >Manifesto
Mozilla</a>
```

- <a> es la **etiqueta**, que al tener contenido se **cierra** con </a>
- "Manifesto Mozilla" es el contenido del **elemento**.
- href="https://www.mozilla.org/es-AR/about/manifesto/" es un **atributo**. href es el nombre del atributo y a continuación del signo "=" escribimos su valor.

Los elementos HTML se pueden anidar, lo que significa que unos elementos pueden contener otros elementos, pero nunca pueden entrelazarse, es decir, no podemos cerrar un elemento si no se han cerrado todos los elementos contenidos en él.

Correcto:

```
<elemento1>
  <elemento2>
    .....
  </elemento2>
</elemento1>
```

Incorrecto:

```
<elemento1>
  <elemento2>
    .....
</elemento1>
  </elemento2>
```

Ejemplo de código HTML correcto con etiquetas anidadas:



```
<ul>
  <li>tecnólogos</li>
  <li>pensadores</li>
  <li>constructores</li>
</ul>
```

## 3.2. COMENTARIOS

Los comentarios son textos explicativos introducidos en un documento que no serán interpretado por navegador. No se visualizará como resultado de ejecución de la página HTML. Los comentarios no se pueden anidar.

```
<!--comentario-->
<!--Esto no aparecerá en la página web.
Aunque ocupe varias líneas.-->
```

## 3.3. ESTRUCTURA DE UNA PÁGINA HTML

La estructura básica mínima de una página web **HTML5 bien formada** necesita de las siguientes etiquetas:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Título de la página</title>
</head>
<body>
  </body>
</html>
```

Todos estos elementos deben de aparecer en nuestras páginas para no tener problemas de validación. Para que un documento HTML sea válido debe de superar el proceso de validación oficial de la W3C, disponible en <http://validator.w3.org/> . El sitio web ofrece tres posibilidades, validar una web a través de su URL, validar un fichero o validar código HTML copiado directamente a la web.

Analicemos las diferentes partes del documento detenidamente.

## 3.4. DECLARACIÓN DEL TIPO DE DOCUMENTO

Permite identificar la DTD (la estructura de etiquetas del documento, los atributos que pueden tener las etiquetas, los eventos, etc.) adecuada para procesar el documento.

En **HTML 4** puede haber tres tipos de DTD, en cada uno de los tipos se indicaba varios tipos de información:

- **DTD Estricta:** la utilizaremos en el caso de que vayamos a cumplir escrupulosamente con esta recomendación, almacenando toda la información sobre el formato en una hoja de estilos CSS.

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Strict//EN"
"http://www.w3.org/TR/html4/strict.dtd">
```

- **DTD de Transición:** permite algunas marcas de presentación y elementos obsoletos, pero no conjuntos de marcos.

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
```

- **DTD con marcos:** la utilizaremos si necesitamos incluir marcos (frame) en nuestro documento. Un marco o frame es una ventana independiente dentro de nuestra ventana principal.

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN"
"http://www.w3.org/TR/html4/frameset.dtd">
```

En HTML 5 la declaración es más simple:

```
<!DOCTYPE html>
```

Es **crucial** y debe ser la primera línea del documento.

## 3.5. ETIQUETA HTML

Es la **etiqueta raíz** de toda la página, se identifica con la etiqueta **<html>** y finaliza por tanto con **</html >**.

El atributo `lang="es"` especifica el idioma principal del contenido de la página (español en este caso), lo cual es importante para la **accesibilidad y los motores de búsqueda**. En HTML5 es **muy aconsejable** definir qué lenguaje o idioma estamos utilizando para los

contenidos (durante el curso será **obligatorio**, penalizando si no se incluye). Podéis ver un listado con los idiomas admitidos en [este enlace](#).

Una sola página completa no admite más de un idioma, pero es posible indicar que una parte de nuestra página está en un idioma diferente al principal. Por ejemplo, para indicar que un determinado párrafo está en inglés tendríamos:

```
<p lang="en"> ... English words ... </p>
```

## 3.6. ETIQUETA HEAD

Contiene **información (metadatos)** general acerca del propio documento y su visualización.

Se identifica con la etiqueta **<head>** y finaliza por tanto con **</head>**. Dentro de **<head>** se pueden emplear diferentes elementos referenciados por sus etiquetas. Los más relevantes son:

### 3.6.1. TITLE

```
<title> [cadena de caracteres] </title>
```

Da título al documento, y en los navegadores normalmente se visualiza en la barra de título. Es fundamental para el SEO (optimización en motores de búsqueda), por lo que es **obligatorio incluir esta etiqueta**.

```
<head>
  <title> Mi primera página</title>
</head>
```

### 3.6.2. META

```
<meta http-equiv="VALOR1" | name="VALOR2" | charset="VALOR3" |
content="VALOR4">
```

**No tienen etiqueta de cierre.** Puede haber **más de un** **<meta>** dentro de **<head>**. Sus posibles atributos son: **http-equiv**, **name**, **charset** y **content**.

Las etiquetas **<meta>** o “meta tags” proporcionan información sobre el documento HTML. Son identificadores ocultos, es decir, instrucciones especiales del lenguaje HTML que no

son mostradas directamente en el navegador, pero que pueden ser utilizadas por navegadores, motores de búsqueda u otros servicios web. Ejemplo:

La etiqueta meta tiene varios atributos con distintas funciones:

### ATRIBUTO CHARSET

Como vimos en la estructura básica de un documento HTML es **obligatorio** que dentro de <head> aparezca una etiqueta <meta> con el atributo **charset**.

Especifica la **codificación de caracteres** del documento. UTF-8 es el estándar recomendado, ya que incluye casi todos los caracteres y símbolos internacionales, evitando problemas con acentos y caracteres especiales como la 'ñ'.

```
<meta charset="UTF-8">
```

### ATRIBUTOS NAME Y CONTENT

Se utilizan para optimizar el resultado en los motores de búsqueda. Contienen información referida al documento HTML. No es información que el usuario ve directamente en la página, pero es crucial para que las máquinas la entiendan correctamente. El atributo **name** actúa como una **etiqueta o identificador** (el tipo de información que vas a proporcionar), y el atributo **content** proporciona el **valor o contenido** de esa información, por lo que casi siempre aparecen juntos, ya que name solo nombra la propiedad, mientras que content le da su valor real. El atributo "name" puede tomar entre otros los siguientes valores:

- **autor:** para indicar el nombre del autor del sitio.  
<meta name="author" content="Rita Maestre García">
- **copyright:** indica si hay información bajo derechos de autor.  
<meta name="copyright" content="Editorial Garceta 2012">
- **date:** para la fecha de creación de la página.  
<meta name="date" content="16 de Octubre de 2022">
- **generator:** para indicar la herramienta utilizada para la creación del sitio.  
<meta name="generator" content="Visual Studio Code">
- **description:** para explicar el contenido y objetivos del sitio.  
<meta name="description" content="Enseñanza del módulo Lenguajes de Marcas">
- **distribution:** para indicar el nivel de distribución que queremos para nuestro sitio. Global (internet), Local (para una red local) e IU (uso interno).  
<meta name="distribution" content="global">

- **keywords:** para indicar palabras clave de nuestro sitio. Esto era muy utilizado por los motores de búsqueda hace un tiempo para indexar resultados pero, debido a su abuso, hoy prácticamente no es tenido en cuenta.

<meta name="keywords" content="Ciclos, Formativos, DAW">

- **locality:** indica al robot de indexación la localidad donde se ubica nuestro sitio.

<meta name="locality" content="Madrid, España">

- **rating:** para recomendar el visionado de la página según la edad, de un modo similar a las películas de cine (general, adult).

<meta name="rating" content="general">

- **revisit-after:** sirve para indicar a los motores de búsqueda que revisiten nuestra página para volverla a indexar porque ha sufrido modificaciones. Los motores de búsqueda suelen tener su propio patrón de retorno a las páginas ya indexadas, por lo que no siempre cumplen con el contenido de esta etiqueta.

<meta name="revisit-after" content="3 days">

- **reply-to:** indica un email para contactar con el administrador del sitio. No es recomendable su uso pues puede servir como herramienta de SPAM.

- **robots:** sirve para controlar los robots de los motores de búsqueda.

<meta name="robots" content="index, nofollow">

Los valores posibles son:

| Valor    | Utilidad                                                                          |
|----------|-----------------------------------------------------------------------------------|
| index    | Indica al robot que indexe la página                                              |
| follow   | Indica al robot que siga los hipervínculos de esta página para indexarlos también |
| all      | Equivale a los dos anteriores                                                     |
| noindex  | No indexa la página                                                               |
| nofollow | No sigue los hipervínculos                                                        |

- **viewport:** Una de las etiquetas <meta> más importantes en el desarrollo web moderno es la del **viewport**. Esta etiqueta es **crucial** para que tu página web se vea correctamente en dispositivos móviles como smartphones y tablets.

Sin ella, los navegadores móviles intentarán mostrar la versión de escritorio de tu página, lo que hará que todo se vea muy pequeño y los usuarios tengan que hacer zoom constantemente para poder leer algo, lo cual va en contra del diseño **responsive** que tiene que tener todo sitio web en la actualidad, esto es, que se adapta a cualquier tipo de pantalla.

La etiqueta viewport le da al navegador instrucciones sobre cómo controlar las dimensiones y la escala de la página. La forma más común y recomendada es la siguiente:

```
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

- **name="viewport"**: Indica que estamos definiendo la configuración del "área visible" del navegador.
- **content="..."**: Contiene las instrucciones.
  - **width=device-width**: Le dice al navegador: "Haz que el ancho de la página sea igual al ancho de la pantalla del dispositivo". De esta manera, la página ocupará todo el ancho disponible, sin importar si es un móvil o un monitor grande.
  - **initial-scale=1.0**: Establece el nivel de zoom inicial cuando la página se carga por primera vez. Un valor de 1.0 significa que no habrá ningún zoom, mostrando la página en su tamaño real (100%).

## ATRIBUTO HTTP-EQUIV

Este atributo se utiliza para especificar información de las cabeceras http que **regulan el diálogo** entre el servidor y el navegador. Los valores establecidos por este metadato pueden ser utilizados por el servidor al entregar la página al navegador del usuario, y son utilizadas para refinar la información y dar instrucciones al navegador que las está leyendo.

El formato es:

```
<meta http-equiv="valor_http-equiv" content="valor_content">
```

Los posibles valores del atributo son:

- **refresh**: sirve para indicar el tiempo de espera en segundos hasta una actualización o redirección automática de la página.

```
<meta http-equiv="refresh" content="10; URL=http://otro\_sitio.html">
```

Aunque **sigue siendo válido** y es utilizable en algunos casos como en páginas de transición, **no es la mejor práctica** en la mayoría de los casos. Se usa cuando no hay acceso al servidor o se necesita una solución rápida, pero para proyectos bien estructurados, se recomienda usar redirecciones HTTP o JavaScript.

- **pragma:** sirve para evitar que la página sea consultada en memoria caché, por lo tanto, el navegador debe solicitarla al servidor cada vez que queramos visualizarla.

```
<meta http-equiv="pragma" content="no-cache">
```

Esta etiqueta **no es tomada en cuenta por navegadores modernos**. Si necesitas controlar la caché, lo mejor es hacerlo desde el servidor con cabeceras HTTP o usar técnicas en JavaScript.

### 3.6.3 LINK

Define un **vínculo a otro documento** indicado por el **atributo href**. No tiene etiqueta de cierre. En temas posteriores lo utilizaremos para hacer referencias a **hojas de estilo CSS** externas a nuestro documento.

Algunos de sus atributos:

- **rel:** este atributo indica la **relación** del documento enlazado con el actual. El uso más común para este atributo es especificar el enlace a una hoja de estilos externa: en este caso el atributo `rel` se establece con valor `stylesheet`
- **type:** este atributo es usado para definir el **tipo de contenido** al que se enlaza. El uso común de este atributo es para definir el tipo de hoja de estilos enlazada, y el valor más común es `text/css`.
- **href:** este atributo especifica la **url** del recurso enlazado. La URL puede ser absoluta o relativa.

```
<link rel="stylesheet" type="text/css"
href="http://mipagina.es/estilos.css">
```

### 3.6.4. SCRIPT

Sirve para insertar **scripts** dentro de un documento. Solo se incorpora en ese caso. Puede aparecer **varias veces**, tanto en la **cabecera** como en el **cuerpo** de nuestro documento. Las etiquetas de inicio y cierre son obligatorias.

Sus atributos:

- **charset:** codificación de caracteres del script indicado por `src`.
- **type:** indica el lenguaje de programación en el que está el script.
- **src:** indica la URI donde está alojado el script externo (del inglés source).
- **defer:** es un booleano, su presencia indica al navegador que el script no va a generar ningún contenido en el documento.

Ejemplo script interno:

```
<script type="text/javascript">
  aquí iría nuestro código en javascript
</script>
```

Ejemplo script externo:

```
<script type="text/javascript" src="/js/fichero.js"></script>
```

### 3.6.5. STYLE

Anteriormente al estudiar la etiqueta `<link>` vimos el modo de insertar estilos CSS a través de un archivo externo. Aunque menos utilizado, también existe la posibilidad de **insertar estilos** directamente **dentro de nuestra página**, lo cual reduce las posibilidades de reutilización y homogeneización, pero puede ser útil en algunos casos.

Para ello utilizaremos la etiqueta `<style>`. Podemos incorporar en ella el atributo “type” que indica el tipo de lenguaje de programación de estilos utilizado, que normalmente será “text/css”.

Ejemplo:

```
<style type="text/css">
  body{
    margin-left:40px;
    margin-top: 40px;
    margin-right: 40px;
  }
</style>
```

Un posible ejemplo que incorporase todas las posibles etiquetas admitidas en `<head>` podría ser:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width,
initial-scale=1.0">

  <title>Mi Primera Página Web | Curso de HTML</title>

  <meta name="description" content="Aprende a crear páginas web desde
cero con nuestro curso completo de HTML5.">
```



```

<meta name="author" content="Nombre del Curso o Autor">

<link rel="stylesheet" href="css/estilos.css">

<link rel="icon" href="favicon.ico" type="image/x-icon">

<style>
    body {
        font-family: sans-serif;
        line-height: 1.6;
    }
</style>

<script src="js/main.js" defer></script>

</head>
<body>
</body>
</html>

```

### 3.7. CONTENIDO DEL CUERPO. ETIQUETA <body>

Es el **contenedor de la información** propia del documento, de nuestra web. Se identifica con la etiqueta **<body>** . Puede presentar algunos atributos:

- Los atributos (los veremos más adelante) tales como class, id o style **sí son admitidos y válidos**.
- Los atributos de **presentación**, background, text o bgcolor, ya **se consideran obsoletos** en HTML moderno y se sustituirán por propiedades CSS3 equivalentes. Los veremos por si los encontráis en vuestra vida profesional, pero **se penalizará su utilización** en ejercicios, tareas o exámenes.

<body [background="url de imagen"] [bgcolor="color"]>

- El atributo background (**OBSOLETO**) se utiliza para poner una imagen de fondo a nuestra página web con la sintaxis:

<body [background="url de imagen"]>

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Etiqueta body</title>
</head>
<!-- El atributo background está obsoleto y NO DEBE SER UTILIZADO -->

```

```
<body
background="./images/abstract_pattern_design_background_2007.jpg">

</body>
</html>
```

- El atributo bgcolor (**OBSOLETO**) permite definir un color para el fondo de la página web:

```
<body [bgcolor="color"]>
```

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Etiqueta body</title>
</head>
<!-- El atributo bgcolor está obsoleto y NO DEBE SER UTILIZADO -->
<body bgcolor="#FF0000"> <!-- #FF0000 = color rojo -->

</body>
</html>
```

- El atributo text (**OBSOLETO**) permite definir el color del texto de la página:

```
<body [text="color"]>
```

Hablemos sobre un tema importante, **COLORES**. Aunque no los vayamos a utilizar en HTML, la forma de usarlos en CSS es similar, por lo que merece la pena hacer un estudio sobre sus notaciones.

- El color en diseño web suele especificarse en modo **RGB (Red Green Blue)**. Los valores RGB se indican en hexadecimal. Para conseguir un color, mezclaremos valores de esta manera: #RRGGBB donde **RR** corresponde a la cantidad de **rojo**, **GG** a la cantidad de **verde** y **BB** a la cantidad de **azul**. Cada valor puede crecer **desde 00 hasta FF**.

Por ejemplo FF0000 es el color rojo. Para indicar que está en hexadecimal antepone el carácter #.

Otros ejemplos:

- Naranja: #FF8000

- Verde turquesa: #339966
- Azul oscuro: #000080

En la siguiente página puedes obtener el código hexadecimal o decimal de cualquier color:

<https://imagecolorpicker.com/>

• Además de en modo RGB, los 16 colores de la paleta VGA Standard pueden ponerse en formato texto (nombre del color en inglés), aunque esto último no es aconsejable porque puede haber problemas de interpretación con los distintos navegadores:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Etiqueta body</title>
</head>
<!-- El atributo bgcolor está obsoleto y NO DEBE SER UTILIZADO -->
<body bgcolor="red"> <!-- red = color rojo -->
</body>
</html>
```

### Nombres de colores y valores sRGB

|                                                                                                                   |                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
|  Black = "#000000" (Negro)     |  Green = "#008000" (Verde)       |
|  Silver = "#C0C0C0" (Plateado) |  Lime = "#00FF00" (Verde lima)   |
|  Gray = "#808080" (Gris)       |  Olive = "#808000" (Verde oliva) |
|  White = "#FFFFFF" (Blanco)    |  Yellow = "#FFFF00" (Amarillo)   |
|  Maroon = "#800000" (Marrón)   |  Navy = "#000080" (Azul marino)  |
|  Red = "#FF0000" (Rojo)        |  Blue = "#0000FF" (Azul)         |
|  Purple = "#800080" (Púrpura)  |  Teal = "#008080" (Azul verdoso) |
|  Fuchsia = "#FF00FF" (Fucsia)  |  Aqua = "#00FFFF" (Celeste)      |

En el **ámbito del cuerpo (body)** del documento se pueden emplear distintos elementos para dar formato al contenido de este (negrita, subrayado, tablas, formularios, ...). Veremos todos estos elementos en los siguientes apartados.

En HTML la mayoría de los elementos se clasifican en **elementos de bloque** o **elementos en línea**.

- **Los elementos de bloque:** son elementos que crean estructuras más grandes, que pueden contener a los elementos de línea y a otros elementos de tipo bloque. Estos elementos siempre comienzan en una nueva línea.



- **Los elementos de línea:** Estos solo pueden contener datos y otros elementos de tipo línea, no comienzan con línea nueva, sino que parecen continuar en la misma línea que otros elementos, siempre que éstos también sean en línea.



### 3.7.1. MANEJO DEL TEXTO

#### 3.7.1.1. ENCABEZADOS

Son elementos de **bloque**. HTML tiene **seis “niveles” de encabezados**:

**<h1> <h2> <h3> <h4> <h5> <h6>**

- Encabezado de nivel 1 para **encabezados principales**: <h1>Texto del encabezado h1</h1>
- Encabezado de nivel 2 para **subcabeceras**: <h2>Texto del encabezado h2</h2>
- Encabezado de nivel 3 para secciones posteriores: <h3>Texto del encabezado h3</h3>
- Encabezado de nivel 4 para secciones posteriores: <h4>Texto del encabezado h4</h4>
- Encabezado de nivel 5 para secciones posteriores: <h5>Texto del encabezado h5</h5>
- Encabezado de nivel 6 para secciones posteriores: <h6>Texto del encabezado h6</h6>

Los navegadores muestran el contenido de las cabeceras en diferentes tamaños, siendo <h1> el mayor y <h6> el más pequeño. Veremos cómo controlar el tamaño, color y fuente del texto desde CSS más adelante.

Ejemplo:

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta name="author" content="CDR">
  <meta name="description" content="Lenguajes_de_marcas">
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Encabezados</title>
</head>

<body>
  <!-- Los encabezados se muestran en diferentes tamaños según su
"nivel" -->
  <h1>Texto del encabezado h1</h1>
  <h2>Texto del encabezado h2</h2>
  <h3>Texto del encabezado h3</h3>
  <h4>Texto del encabezado h4</h4>
  <h5>Texto del encabezado h5</h5>
  <h6>Texto del encabezado h6</h6>
</body>

</html>
```

Cuya visualización en un navegador sería:

Texto del encabezado h1

Texto del encabezado h2

Texto del encabezado h3

Texto del encabezado h4

Texto del encabezado h5

Texto del encabezado h6

Admiten el atributo (**OBSOLETO**) `align=(left|center|right|justify)` para definir la alineación del texto dentro del bloque, aunque está obsoleto por lo se recomienda usar CSS.

### 3.7.1.2. PÁRRAFOS

Son elementos de **bloque**.

`<p> . . </p>`

Hace que el texto que está contenido en esta etiqueta sea considerado un párrafo, como un párrafo en Word. A ese texto se le colocará un salto de carro al final y se mostrará un pequeño espacio entre el párrafo y los siguientes elementos.

La longitud de las líneas viene definida por el tamaño de la ventana del navegador, el navegador se encarga de que la línea ocupe todo el ancho disponible, en caso de que no quepa, el texto se ajusta ocupando más o menos líneas.

Ejemplo:

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta name="author" content="CDR">
  <meta name="description" content="Lenguajes_de_marcas">
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Párrafos</title>
</head>
```

```
<body>
  <p>Esto es un ejemplo de un párrafo en el que hay una línea</p>
  <p>Esto es un ejemplo de un párrafo en el que hay una línea muy
larga y el navegador automáticamente la adapta a varias líneas. Esto es
un ejemplo de un párrafo ...</p>
  <p>Este es el último párrafo</p>
</body>

</html>
```

Cuya visualización sería:

---

Esto es un ejemplo de un párrafo en el que hay una línea

Esto es un ejemplo de un párrafo en el que hay una línea muy larga y el navegador automáticamente la adapta a varias líneas. Esto es un ejemplo de un párrafo ...

Este es el último párrafo

---

Admite el atributo (**OBSOLETO**) `align=(left|center|right|justify)` para definir la alineación del texto dentro del bloque.

### 3.7.1.3. SALTOS DE LÍNEA

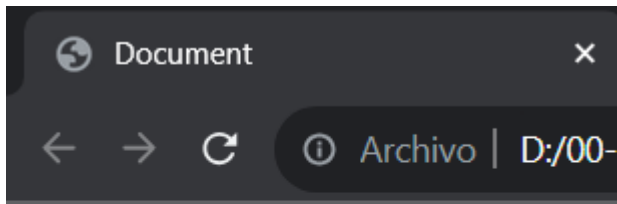
El navegador mostrará automáticamente cada párrafo en una nueva línea, pero si quieres **añadir un salto de línea** dentro de un párrafo puedes utilizar el elemento vacío (sin etiqueta de cierre):

**<br>**

Los elementos `<br>` deben usarse sólo para los saltos de línea que son realmente **parte del contenido**, como en poemas o direcciones. El siguiente ejemplo se considera un uso correcto de `<br>`:

```
<body>
<p> María Gómez <br> Calle Bravo Murillo no152 <br> 28020 Madrid </p>
</body>
```

Cuya visualización sería:



María Gómez  
Calle Bravo Murillo nº152  
28020 Madrid

Los elementos `<br>` **no deben usarse para separar grupos temáticos** en un párrafo. Los siguientes ejemplos no son conformes, ya que abusan del elemento `<br>` (en este ejemplo hay etiquetas que se explicarán más adelante):

```
<p>  
  <a href="#"> 34 comentarios. </a> <br>  
    <a href="#"> Agrega un comentario. </a>  
</p>  
<p> <label> Nombre: <input name="name"> </label> <br>  
    <label> Dirección: <input name="address"> </label>  
</p>
```

La alternativa válida sería sustituir los elementos `<br>` por párrafos que separen el contenido en líneas independientes.

#### 3.7.1.4. LÍNEA HORIZONTAL

Representa un **cambio de tema** entre párrafos (por ejemplo, un cambio de sección). En versiones previas de HTML representaba una línea horizontal y aún puede usarse como tal, pero tiene más un **significado semántico** que visual. Es también un elemento vacío, por lo que no tiene etiqueta de cierre.

**`<hr>`**

Ejemplo:

```
<!DOCTYPE html>  
<html lang="es">  
  
<head>  
  <meta name="author" content="CDR">  
  <meta name="description" content="Lenguajes_de_marcas">  
  <meta charset="UTF-8">
```



```

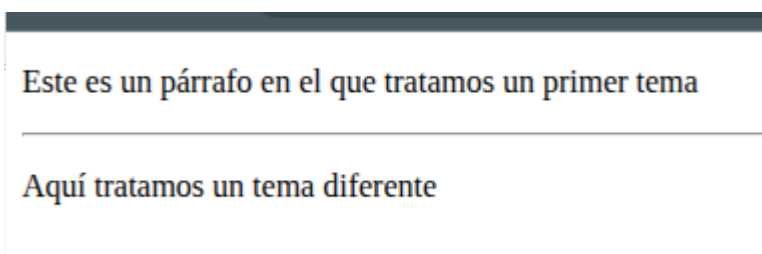
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Líneas horizontales</title>
</head>

<body>
  <p>Este es un párrafo en el que tratamos un primer tema</p>
  <hr>
  <p>Aquí tratamos un tema diferente</p>
</body>

</html>

```

Visualización:



Admite los siguientes atributos (**OBSOLETOS, en diseño moderno se sustituyen por CSS**):

- `width="tamaño"`. Anchura de la línea. Puede ser relativa (en porcentaje, p.e. 50%) o absoluta (en píxeles, p.ej. 15 px).
- `align="alineación"`. Indica cómo se alinearán la línea. (Left, Center, Right).
- `size="n"` Grosor de la línea en píxeles.

### 3.7.1.5. ESTILOS DE FUENTE

Todos los siguientes elementos son el línea, por lo que se escribirán a continuación del anterior. Todas tienen etiqueta de apertura y cierre, pueden ir en el interior de un párrafo y son combinables entre sí.

- **Negrita: <b>**
- No debe implicar la adición de significado, sino únicamente resaltar un texto sin implicar énfasis o importancia especial. No puede utilizarse con fines de presentación, por lo que no se utilizará, por ejemplo, para resaltar un título.

```

<p>Los valores <b>brutos</b> deben compararse con los <b>netos</b>.</p>

```

- **Itálica o cursiva: <i>**
- Se utiliza para indicar texto con tono alternativo (ej. términos técnicos, extranjeros o técnicos). No debe de utilizarse con fines estéticos.

```
<p>La palabra <i>zeitgeist</i> proviene del alemán y significa "espíritu del tiempo".</p>
```

```
<p>Mi novela favorita es <i>La sombra del viento</i> de Carlos Ruiz Zafón.</p>
```

- **<sup>: Superíndice:** Se utiliza para representar exponentes (fórmulas matemáticas), notas al pie, ...
- **<sub>: Subíndice:** Normalmente usado en fórmulas químicas o matemáticas.
- **subrayado: <u>** Se usa para **resaltar visualmente** algo que no tiene énfasis semántico (a diferencia de <em> o <strong>, que serán estudiadas más adelante). En este caso, se subraya el nombre del botón para facilitar la identificación, **sin implicar urgencia o importancia especial**.

```
<p>Haz clic en el botón <u>Confirmar</u> para continuar con el proceso.</p>
```

A partir de aquí los elementos explicados añaden **información extra** a las páginas, dan un **significado** al contenido, por lo que corresponderían a un marcado **semántico**. Los navegadores frecuentemente varían el modo en el que se visualizan estos elementos, pero no deben utilizarse con fines estéticos. Su sentido es describir el contenido de tus páginas web de forma más precisa. Otros programas, como lectores de pantalla, pueden utilizar esta información extra, por ejemplo añadiendo un especial énfasis en palabras contenidas en una etiqueta <em>.

- **tamaño pequeño: <small>** Se usa para mostrar información secundaria o aclaratoria, como notas legales, advertencias suaves, o detalles complementarios. No es solo para hacer el texto más pequeño visualmente, sino para indicar que su importancia es menor.

```
<p>Este producto incluye garantía de 2 años. <small>Aplican condiciones especiales para productos reacondicionados.</small></p>
```

- **tachado: <s>** Indica que el texto **ya no es válido pero no debe ser eliminado**, como precios antiguos, como un precio antiguo durante una oferta. No se usa para tachar por estilo, sino para **comunicar que algo ha sido reemplazado o invalidado**.

```
<p>Precio original: <s>49,99 €</s> Oferta actual: 29,99 €</p>
```

- **<em>: Indica énfasis leve (emphasis).** Usualmente los navegadores usan la cursiva.
- **<strong>: Indica un énfasis más fuerte, como urgencia o relevancia.** Por defecto los navegadores suelen marcar el texto en negrita.

```
<p>Este juguete contiene piezas pequeñas y <strong>no debe ser  
utilizado por niños menores a cinco años</strong></p>
```

- **<cite>**: Se utiliza para nombres de libros, películas, .... dentro de un texto.

```
<p><cite>El grito</cite> de Edward Munch. Pintado en 1893.</p>
```

- **<code>**: Designa un fragmento de código de computadora. Aparece en fuente monoespaciada.
- **<samp>**: Designa una muestra de la salida de un programa, script, etc.
- **<kbd>**: Indica texto que debe ser introducido por el usuario (lo que se debe de teclear, como por ejemplo Ctrl + s)..
- **<var>**: Indica que el texto es una variable o argumento de un programa. Se utiliza en expresiones matemáticas o código.
- **<q>**: el contenido es una cita textual **corta** de una frase. Los navegadores suelen añadir comillas automáticamente.
- **<del>**: marca las partes de un texto que hayan sido suprimidas (tachado) en una revisión del texto. Se puede incluir un atributo **datetime** con la fecha del cambio y el autor.

```
<p>  
  La fecha final de entrega ha cambiado del  
  <del datetime="2024-01-03T12:00:00Z">15 de enero de 2024</del>  
  a 30 de enero de 2024.  
</p>
```

- **<ins>**: el texto afectado ha sido añadido en esta versión del documento (subrayado). Puede incluir el atributo **datetime**.

```
<p>  
  Este blog trata sobre conceptos <del>matemáticos</del>  
  <ins datetime="2018-11-21T15:55:03Z">  
    científicos  
  </ins> a través de artículos de divulgación  
</p>
```

- **<mark>**: Representa texto resaltado con propósitos de referencia, es decir por su relevancia.
- **<abbr>**: Si usas una abreviatura o un acrónimo, puedes usar el elemento **<abbr>**. El atributo **title** en la etiqueta de apertura se utiliza para especificar el término completo.

```
<p>El <abbr title="Doctor">Dr.</abbr> Stephen Vincent Strange es un  
personaje ficticio que aparece en los cómics estadounidenses publicados  
por Marvel Comics.</p>
```

```
<p>La <abbr title="National Aeronautics and Space  
Administration">NASA</abbr> se dedica a la exploración espacial.</p>
```

- **<blockquote>**: El elemento `<blockquote>` se utiliza para citas largas que ocupan un párrafo completo. Observa cómo el elemento `<p>` se sigue utilizando dentro del elemento `<blockquote>`. Los navegadores tienden a sangrar el contenido del elemento `<blockquote>`; sin embargo, no debes usar este elemento solo para sangrar un fragmento de texto; debes lograr este efecto mediante CSS. El atributo **cite** especifica una URL a un documento que explica la cita o por qué se insertó o modificó el texto.

```
<blockquote cite="http://en.wikipedia.org/wiki/Winnie-the-Pooh">
  <p>¿Deberías alguna vez parar de pensar, olvidar y empezar de
nuevo?</p>
</blockquote>
```

- **<address>**: Se utiliza para englobar información de contacto o dirección en un documento HTML. Generalmente, se utiliza para incluir detalles de contacto de una persona o entidad, como su dirección de correo electrónico, número de teléfono, dirección postal, entre otros. Esta etiqueta no se limita exclusivamente a direcciones, sino que puede usarse para cualquier tipo de información de contacto.

```
<address>
  <p><a href="mailto:correo@correo.com">correo@correo.com</a></p>
  <p>Tel.: (+34) 600000000</p>
</address>
```

Encontrarás similitudes en algunas etiquetas como [`<b>`, `<i>`, `<u>`] y [`<strong>`, `<em>`, `<ins>`] ya que visualmente el resultado es el mismo. Las últimas están enfocadas a reflejar un significado semántico y da un sentido concreto al texto afectado. Las primeras son meramente visuales.

Veamos algún ejemplo de uso de estas etiquetas:  
Ejemplo combinando todos:

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta name="author" content="CDR">
  <meta name="description" content="Lenguajes_de_marcas">
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Estilos de fuente</title>
</head>
```

```

<body>
  <p>Los valores <b>brutos</b> deben compararse con los
<b>netos</b>.</p>
  <hr>
  <p>La palabra <i>zeitgeist</i> proviene del alemán y significa
"espíritu del tiempo".</p>
  <hr>
  <p>Mi novela favorita es <i>La sombra del viento</i> de Carlos Ruiz
Zafón.</p>
  <hr>
  <p>Este producto incluye garantía de 2 años. <small>Aplican
condiciones especiales para productos reacondicionados.</small></p>
  <hr>
  <p>Haz clic en el botón <u>Confirmar</u> para continuar con el
proceso.</p>
</body>

</html>

```

---

Los valores **brutos** deben compararse con los **netos**.

---

La palabra *zeitgeist* proviene del alemán y significa "espíritu del tiempo".

---

Mi novela favorita es *La sombra del viento* de Carlos Ruiz Zafón.

---

Este producto incluye garantía de 2 años. Aplican condiciones especiales para productos reacondicionados.

---

Haz clic en el botón Confirmar para continuar con el proceso.

```

<!DOCTYPE html>
<html lang="es">

<head>
  <meta name="author" content="CDR">
  <meta name="description" content="Lenguajes_de_marcas">
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Estilos de fuente</title>
</head>

```

<body>

<h1>Introducción a HTML5 y programación</h1>

<p>En este artículo aprenderás conceptos básicos sobre HTML5, programación y notación científica.</p>

<p>La fórmula del agua es H<sub>2</sub>O y la del dióxido de carbono es CO<sub>2</sub>. En matemáticas,  $3^2$  equivale a 9.</p>

<p>Es <em>muy importante</em> entender la diferencia entre etiquetas de presentación y etiquetas semánticas. De hecho, <strong>HTML5 prioriza el significado sobre el estilo</strong>.</p>

<p>Según <cite>HTML5: The Language of the Web</cite>, el uso correcto de etiquetas mejora la accesibilidad y el SEO.</p>

<p>Para imprimir "Hola mundo" en Python, puedes usar la siguiente instrucción:</p>

<pre><code>print("Hola mundo")</code></pre>

<p>La salida esperada sería:</p>

<pre><samp>Hola mundo</samp></pre>

<p>Para ejecutar el programa, escribe <kbd>python hola.py</kbd> en la terminal.</p>

<p>En la función <code>calcularArea</code>, el parámetro <var>radio</var> representa el radio del círculo.</p>

<p>Como dijo Tim Berners-Lee: <q>El poder de la Web está en su universalidad.</q></p>

<p>La versión anterior decía: <del>Las etiquetas HTML no tienen significado semántico.</del></p>

<p>Ahora se ha corregido por: <ins>Las etiquetas HTML pueden tener significado semántico y funcional.</ins></p>

```
<p><small>Nota: Este artículo es solo para fines educativos. No  
representa documentación oficial.</small></p>
```

```
<p>Recuerda que <mark>las etiquetas semánticas mejoran la  
accesibilidad</mark> y ayudan a los motores de búsqueda a  
entender mejor tu contenido.</p>
```

```
</body>
```

```
</html>
```

Con la siguiente salida:

---

## Introducción a HTML5 y programación

En este artículo aprenderás conceptos básicos sobre HTML5, programación y notación científica.

La fórmula del agua es  $H_2O$  y la del dióxido de carbono es  $CO_2$ . En matemáticas,  $3^2$  equivale a 9.

Es *muy importante* entender la diferencia entre etiquetas de presentación y etiquetas semánticas. De hecho, **HTML5 prioriza el significado sobre el estilo**.

Según *HTML5: The Language of the Web*, el uso correcto de etiquetas mejora la accesibilidad y el SEO.

Para imprimir "Hola mundo" en Python, puedes usar la siguiente instrucción:

```
print("Hola mundo")
```

La salida esperada sería:

```
Hola mundo
```

Para ejecutar el programa, escribe `python hola.py` en la terminal.

En la función `calcularArea`, el parámetro *radio* representa el radio del círculo.

Como dijo Tim Berners-Lee: "El poder de la Web está en su universalidad."

La versión anterior decía: ~~Las etiquetas HTML no tienen significado semántico.~~

Ahora se ha corregido por: Las etiquetas HTML pueden tener significado semántico y funcional.

Nota: Este artículo es solo para fines educativos. No representa documentación oficial.

Recuerda que **las etiquetas semánticas mejoran la accesibilidad** y ayudan a los motores de búsqueda a entender mejor tu contenido.

Las siguientes etiquetas están consideradas **OBSOLETAS** y no deben ser usadas.

- `<big>` tamaño grande (**OBSOLETO, utilizar CSS**).
- `<strike>` tachado (**OBSOLETO, utilizar CSS**).
- `<tt>` texto con apariencia de teletipo (**OBSOLETO, utilizar CSS**).

### 3.7.1.6. TEXTO PREFORMATEADO

Cuando un navegador encuentra dos o más espacios en blanco seguidos los mostrará como uno solo. De igual modo ocurre con los saltos de línea, que también son tratados como un espacio. Esto puede ser utilizado por los diseñadores para indentar su código para hacerlo más legible sin afectar a la forma en la que la información es mostrada.

Si queremos que se respete el modo en el que la información está escrita, utilizaremos la etiqueta **<pre>** que sirve para indicarle al navegador que respete todos los formatos de texto que se encuentren dentro de esta etiqueta, respetando tabuladores, espacios y saltos de línea. Etiquetas de apertura y cierre son obligatorias.

**NO** es para crear **espaciado o márgenes ni para indentar un párrafo normal**, únicamente se utiliza para mostrar **texto preformateado**. Algunos ejemplos válidos de uso son:

- **Bloques de Código:** Este es el uso más común. La indentación y los saltos de línea son vitales para que el código sea legible.
- **Poesía y Letras de Canciones:** La estructura en versos y estrofas es fundamental, y **<pre>** la preserva perfectamente sin necesidad de usar etiquetas **<br>** para cada línea.
- **Arte ASCII:** Cualquier dibujo hecho con caracteres de texto necesita que se respete la alineación exacta de cada carácter.

```
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <code>
    <pre>
function saludar(nombre) {
  console.log("Hola, " + nombre);
}
    </pre>
  </code>
  <pre>
Me gusta cuando callas
porque estás como ausente,
y me oyes desde lejos,
y mi voz no te toca.
  </pre>
  <pre>
(\_/)
(='. ')=)
  </pre>
  </code>
</body>
</html>
```



```
(")_(")
</pre>
</body>

</html>
```

La salida será:

---

```
function saludar(nombre) {
  console.log("Hola, " + nombre);
}
```

```
Me gusta cuando callas
porque estás como ausente,
y me oyes desde lejos,
y mi voz no te toca.
```

```
(\_/)
(='.'=)
(")_(")
```

### 3.7.2. LISTAS

Hay múltiples ocasiones en las que necesitaremos una **lista** para mostrar información al usuario de nuestras aplicaciones. HTML ofrece a los autores varios mecanismos para especificar listas de información. Los **tipos** de listas son los siguientes:

- Las **listas ordenadas** son listas donde cada elemento está numerado. Por ejemplo, la lista podría ser un conjunto de pasos para una receta que deben realizarse en orden, o un contrato legal donde cada punto debe identificarse con un número de sección.
- Las **listas no ordenadas** son listas que comienzan con una viñeta o símbolo (en lugar de caracteres que indican orden).
- Las **listas de definiciones** se componen de un conjunto de términos junto con sus definiciones.

#### 3.7.2.1. LISTAS DE ELEMENTOS ORDENADAS

En una **lista ordenada** cada elemento de la lista comienza por un **número o elemento que indica orden**, igual a su posición en la lista. Corresponde con el elemento **<ol>**.

Su contenido está formado únicamente por elementos `<li>`. Los elementos `<li>` son los que contienen el texto de cada uno de los ítems que componen la lista. Las listas pueden anidarse.

Atributos permitidos:

- **start**: indica el número del primer elemento de la lista. Válido para los tipos numéricos. Esto es muy útil cuando necesitas continuar una lista que ha sido interrumpida por otro contenido, como un párrafo o una imagen. Su uso no es decorativo, sino que afecta a la **estructura y significado** de la lista.
- **type**: indica el tipo de secuencia de la lista y puede tomar los valores 1 (números decimales), a (letras minúsculas), A (letras mayúsculas), i (números romanos minúsculas) I (números romanos mayúsculas). Su uso está **FUERTEMENTE DESACONSEJADO** y utilizaremos propiedades CSS para sustituirlo.

**<li> Elemento de lista:** Es el objeto que contiene el texto de la lista, puede tener otros elementos de bloque como párrafos en su interior o directamente el texto.

Admite el atributo **value** para indicar el número del objeto de la lista actual. No es de uso común y debe evitarse para fines puramente visuales, pero es válido si la numeración del contenido lo requiere, como en documentos legales donde la numeración de los artículos no es consecutiva

### 3.7.2.2. LISTAS NO ORDENADAS

Lista de textos, cada entrada de la lista comienza por un carácter de señalización (bullet). Las etiquetas de apertura y de cierre son obligatorias. Corresponde con el elemento `<ul>`.

Su contenido está formado únicamente por elementos `<li>`. Los elementos `<li>` son los que contienen el texto. Las listas pueden anidarse.

Atributos permitidos:

- **type**: Indica qué tipo de bullet se usará “circle” (círculo), “disc” (disco) y “square” (cuadrado). Su uso está **FUERTEMENTE DESACONSEJADO** y utilizaremos propiedades CSS para sustituirlo.

**<li> Elemento de lista:** Es el objeto que contiene el texto de la lista, puede tener otros elementos de bloque como párrafos en su interior o directamente el texto.

Ejemplo combinando ambas:

```
<html lang="en">

<head>
  <meta charset="UTF-8">
```

```

<meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <p>Lista de <b>ingredientes</b>:</p>
  <ul>
    <li>1 kg de patatas King Edward</li>
    <li>100 ml de leche</li>
    <li>50 g de mantequilla con sal</li>
    <li>Nuez moscada recién rallada</li>
    <li>Sal y pimienta al gusto</li>
  </ul>
  <p>Receta:</p>
  <ol>
    <li>Cortar las patatas en cuartos</li>
    <li>Cocer a fuego lento en agua con sal durante 15-20 minutos
hasta que estén tiernas</li>
    <li>Calentar la leche, la mantequilla y la nuez moscada</li>
    <li>Ecurrir las patatas y machacarlas</li>
    <li>Añadir la mezcla de leche</li>
  </ol>
</body>

</html>

```

Como hemos indicado anteriormente, las listas pueden anidarse. Dado el siguiente ejemplo:

```

<body>
  <h1>Listas anidadas</h1>
  <hr>
  <h2>Tecnologías</h2>
  <ol>
    <li><b>Televisores</b>
      <ul>
        <li>Sony</li>
        <li>Samsung</li>
      </ul>
    </li>
    <li><b>Teléfonos móviles</b>
      <ul>

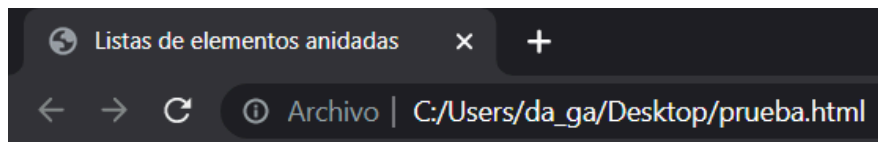
```

```

        <li>Samsung</li>
        <li>Xiaomi</li>
    </ul>
</li>
<li><b>Carcasas móviles</b>
    <ol>
        <li>Originales</li>
        <li>Falsas</li>
    </ol>
</li>
</ol>
</body>

```

Genera la siguiente salida:



# Listas anidadas

---

## Tecnología

1. **Televisores**
  - Sony
  - Samsung
2. **Teléfonos móviles**
  - Samsung
  - Xiaomi
3. **Carcasas móviles**
  1. Originales
  2. Falsas

### 3.7.2.3. LISTAS DE DEFINICIONES

#### <dl>

Son listas que no tienen índices ni símbolos como las anteriores. Estas listas se componen de **dos elementos: el término y la definición**, a modo de **diccionario** (no solo para definiciones, también para glosarios, explicaciones técnicas, ...). Todo su texto aparece con sangría.

Se construyen con elementos **<dl>** y en su interior solo hay elementos **<dt>** y **<dd>**. Las etiquetas apertura y cierre son obligatorias.

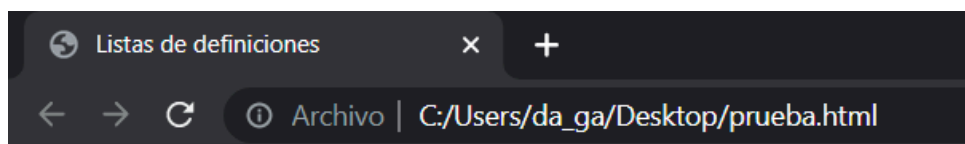
**<dt>**: Es el **término** que se quiere definir, suele ser una palabra o un texto breve. Su etiqueta de cierre es opcional pero se recomienda usarla.

**<dd>**: Es la **definición** del término anterior, suele ser un texto largo y se trata de un elemento en bloque. Su etiqueta de cierre es opcional pero se recomienda usarla. A veces se pueden ver dos términos que utilizan la misma definición o dos diferentes definiciones para el mismo término.

Dado el siguiente ejemplo:

```
<body>
  <h1>Lenguajes de marcas</h1>
  <hr>
  <dl>
    <dt>HTML</dt>
    <dd>HyperText Markup Language</dd>
    <dd>Lenguaje de Marcas de Hipertexto</dd>
    <dt>XHTML</dt>
    <dd>Extensible HyperText Markup Language</dd>
  </dl>
</body>
```

La visualización en un navegador sería:



# Lenguajes de marcas

---

HTML

HyperText Markup Language

Lenguaje de Marcas de Hipertexto

XHTML

Extensibel HyperText Markup Language

### 3.7.3. ENLACES

Los **enlaces** son la característica distintiva de la web, ya que permiten saltar de una página a otra, lo que facilita la navegación.

Es común encontrar los siguientes **tipos** de enlaces:

- Enlaces de un sitio web a otro
- Enlaces de una página a otra dentro del mismo sitio web
- Enlaces de una parte de una página web a otra parte de la misma página
- Enlaces que se abren en una nueva ventana del navegador
- Enlaces que inician tu programa de correo electrónico y dirigen un nuevo correo electrónico a alguien

Los enlaces se crean mediante el elemento `<a>`. Los usuarios pueden hacer clic en cualquier elemento entre la etiqueta de apertura `<a>` y la de cierre `</a>`. El atributo **href** especifica la página a la que desea enlazar.

```
<a href="http://www.imdb.com">IMDB</a>
```

El texto entre la etiqueta de apertura `<a>` y la de cierre `</a>` se conoce como **texto del enlace**. Siempre que sea posible, el texto del enlace debe explicar a dónde se dirigirá a los visitantes si hacen clic en él (en lugar de simplemente decir "haga clic aquí").

Un texto de enlace claro puede ayudar a los visitantes a encontrar lo que buscan. Esto les dará una impresión más positiva de tu sitio y puede animarlos a visitarlo durante más tiempo. (También ayuda a quienes usan lectores de pantalla).

Para escribir un buen texto de enlace, piensa en palabras que la gente podría usar al buscar la página a la que enlaza (Por ejemplo, en lugar de escribir "lugares para alojarse", podrías usar algo más específico como "hoteles en Nueva York").

El siguiente atributo **title** es el encargado de ofrecer una descripción emergente del enlace.

#### 3.7.3.1. ENLACES A OTROS SITIOS

Al enlazar a **otro sitio web**, el valor del atributo `href` será la dirección web completa del sitio, conocida como **URL absoluta**. Los navegadores muestran por defecto los enlaces de color azul y subrayados. Veremos cómo cambiar esto utilizando CSS.

**URL** (Uniform Resource Locator) significa Localizador Uniforme de Recursos. Cada página web tiene su propia URL. Esta es la dirección web que escribirías en un navegador para visitar esa página específica. Una **URL absoluta** comienza con el nombre de **dominio** de ese sitio y puede ir seguida de la ruta a una **página específica**. Si **no se especifica** ninguna página, el sitio mostrará la **página de inicio (normalmente es la página del sitio llamada `index.html`)**.

Es habitual utilizar el atributo **target** en este tipo de enlaces para que sean abiertos en una nueva ventana. En este caso el valor del atributo es `_blank`. Una de las razones más comunes por las que un autor de páginas web podría querer que un enlace se abra en una nueva ventana es si apunta a otro sitio web. En estos casos, esperan que el usuario regrese a la ventana que contiene su sitio después de terminar de visitar el otro.

### Buenas prácticas: Seguridad en enlaces con `target="_blank"`

Se recomienda añadir siempre `rel="noopener noreferrer"` a estos enlaces.

- **noopener**: Evita que la nueva página pueda acceder y manipular la página original a través del objeto `window.opener` de JavaScript, previniendo ataques de phishing.
- **noreferrer**: Evita que la nueva página sepa de dónde viene el tráfico.

```
<a href="http://www.imdb.com" target="_blank" rel="noopener  
noreferrer">IMDB</a>
```

### 3.7.3.2. ENLACES A OTRAS PÁGINAS DEL MISMO SITIO

Al enlazar a otras páginas del **mismo sitio**, **no es necesario especificar el nombre de dominio** en la URL. Se puede usar una abreviatura conocida como **URL relativa**.

Si todas las páginas del sitio se encuentran en la misma carpeta, el valor del atributo `href` es simplemente el nombre del archivo.

Si hay diferentes páginas de un sitio en carpetas diferentes, se puede usar una sintaxis un poco más compleja para indicar la ubicación de la página en relación con la página actual.

Cuando navegamos por páginas del mismo sitio abriremos los enlaces en la **misma ventana**, por lo que **no es necesario** utilizar el atributo **target** en este tipo de enlaces para que sean abiertos en una nueva ventana. En este caso el valor del atributo es `_self`, pero no se indica porque es el valor por defecto.

#### URL relativas

Al enlazar a otras páginas dentro del mismo sitio, puedes usar URL relativas. Son como una versión abreviada de las URL absolutas, ya que no necesitas especificar el nombre de dominio. Las URL relativas son útiles al crear un sitio web en tu ordenador, ya que puedes crear enlaces entre páginas sin tener que configurar tu nombre de dominio ni tu alojamiento web.

En sitios web grandes, conviene organizar el código colocando las páginas de cada **sección** del sitio en una **carpeta nueva**. Las carpetas de un sitio web a veces se denominan directorios.

A continuación vemos la estructura de directorios de un sitio ficticio. La primera carpeta de nivel más alto es conocida como carpeta **raíz** (en el ejemplo examplearts). A partir de ahí se crean el resto de carpetas y ficheros que forman el sitio.

Cada sección se organiza en carpetas independientes, lo cual ayuda a organizar el sitio. En cuanto a la **estructura** existen **relaciones** entre las carpetas. La carpeta **examplearts** es la **carpeta padre** de las carpetas **movies**, **music** y **theater**. Éstas, a su vez, son **hijos** de la carpeta **examplearts**.

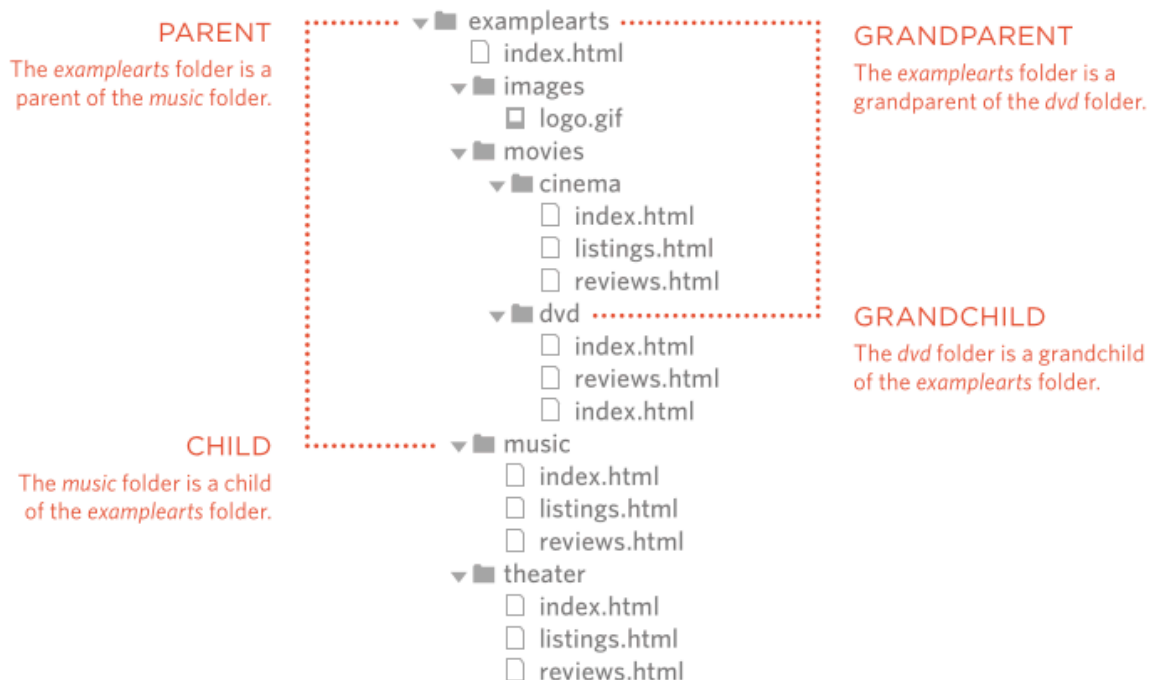
La página principal de un sitio escrito en HTML es llamada index.html. Está será la página de inicio (landing page) de nuestro sitio si únicamente se indica su dominio. Así, si en el navegador escribimos:

*examplearts.com*

la página devuelta será:

*examplearts.com/index.html*

Por su lado, si escribimos *examplearts.com/music* se devolverá la página *examplearts.com/music/index.html*.





Cada página e imagen de un sitio web tiene una URL (o Localizador Uniforme de Recursos). La URL se compone del nombre de dominio seguido de la ruta a esa página o imagen. La ruta a la página de inicio de este sitio es `www.examplearts.com/index.html`. La ruta al logotipo del sitio es `examplearts.com/images/logo.gif`. Las URL se utilizan para enlazar a otras páginas web y para incluir imágenes en el propio sitio. En la siguiente página, encontrará una forma abreviada de enlazar a archivos de su sitio.

Al enlazar a una página de su propio sitio web, no necesitas especificar el nombre de dominio. Puede usar URL relativas, que son una forma abreviada de indicar al navegador la ubicación de una página en relación con la página actual.

Si todos los archivos de tu sitio web están en una sola carpeta, simplemente usa el nombre de archivo de esa página. Si tu sitio web está organizado en carpetas (o directorios) independientes, debes indicarle al navegador cómo acceder desde la página en la que se encuentra a la página a la que enlazas.

Ejemplo de tipo de enlace relativo (del diagrama anterior):

### **Misma carpeta**

Para enlazar a un archivo en la misma carpeta, simplemente use el nombre del archivo. (No se necesita nada más). Para enlazar a reseñas musicales desde la página principal:

```
<a href="reviews.html">Reseñas</a>
```

### **Carpeta secundaria hija**

Para una carpeta secundaria, use el nombre de la carpeta secundaria, seguido de una barra diagonal y el nombre del archivo. Para enlazar a listados de música desde la página principal:

```
<a href="music/listings.html">Listados</a>
```

### **Carpeta secundaria nieta**

Use el nombre de la carpeta secundaria, seguido de una barra diagonal, el nombre de la carpeta secundaria siguiente, otra barra diagonal y el nombre del archivo. Para enlazar a reseñas de DVD desde la página principal:

```
<a href="movies/dvd/reviews.html">Reseñas</a>
```

### **Carpeta padre**

Use `../` para indicar la carpeta superior a la actual y, a continuación, el nombre del archivo. Para enlazar a la página principal desde las reseñas de música:

```
<a href="../index.html">Inicio</a>
```

## Carpeta abuelo

Repite el ../ para indicar que quieres subir dos carpetas (en lugar de una) y, a continuación, escribe el nombre del archivo. Para enlazar a la página principal desde las reseñas de DVD:

```
<a href="../../index.html">Inicio</a>
```

### 3.7.3.3. ENLACES A UN SITIO ESPECÍFICO DE LA MISMA PÁGINA

En la parte superior de una página larga, podrías añadir una lista de contenidos que enlace a las secciones correspondientes más abajo. O podrías añadir un enlace desde la parte inferior de la página hacia la parte superior para evitar que los usuarios tengan que volver a desplazarse hacia arriba.

Antes de enlazar a una parte específica de la página, debes identificar los puntos de la página a los que dirigirá el enlace. Esto se hace mediante el **atributo id** (que se puede usar en todos los elementos HTML). Puedes ver que los elementos <h1> y <h2> de este ejemplo tienen atributos id que identifican esas secciones de la página.

El valor del atributo id debe comenzar con una **letra o un guión bajo** (no un número ni ningún otro carácter) y, **en una misma página, no hay dos atributos id que tengan el mismo valor**.

Para enlazar a un elemento que usa un atributo id, se usa de nuevo el elemento <a>, pero el valor del atributo href comienza con el símbolo #, **seguido del valor del atributo id** del elemento al que se desea enlazar.

En este ejemplo, <a href="#top"> enlaza al elemento <h1> en la parte superior de la página, cuyo atributo id tiene el valor top.

```
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <h1 id="top">Términos cinematográficos</h1>
  <a href="#arc_shot">Plano de arco</a><br />
  <a href="#interlude">Interludio</a><br />
  <a href="#prologue">Prólogo</a><br /><br />
```

```

<h2 id="arc_shot">Plano de arco</h2>
<p>Un plano en el que el sujeto es fotografiado por una cámara
envolvente o en movimiento</p>
<h2 id="interlude">Interludio</h2>
<p>Una breve escena o secuencia intermedia, no específicamente
vinculada a la trama, que aparece dentro de una
    película</p>
<h2 id="prologue">Prólogo</h2>
<p>Un discurso, prefacio, introducción o escena breve que precede a
la acción o trama principal de una película;
    contraste con el epílogo</p>
<p><a href="#top">Arriba</a></p>
</body>

</html>

```

### 3.7.3.4. ENLACES A UN SITIO ESPECÍFICO DE OTRA PÁGINA

Si desea enlazar a una parte específica de una página diferente (ya sea de su propio sitio web o de otro), puede usar una técnica similar.

Siempre que la página a la que enlaza tenga atributos id que identifiquen partes específicas de la página, puede simplemente agregar la misma sintaxis al final del enlace de esa página.

Por lo tanto, el atributo href contendrá la dirección de la página (ya sea una URL absoluta o relativa), seguida del símbolo # y del valor del atributo id utilizado en el elemento al que enlaza.

Por ejemplo, para enlazar a la parte inferior de la página principal del sitio web de un libro sobre HTML y CSS, escribiría:

```
<a href="http://www.htmlandcssbook.com/#bottom">
```

### 3.7.3.5. PROTOCOLOS ESPECIALES EN ENLACES

A la hora de referenciar el destino de nuestro enlace en el atributo href podemos introducir protocolos especiales en caso de enlazar con una dirección de correo electrónico y un número de teléfono:

- **Enlaces de correo:** En este caso el navegador tratará de abrir un gestor de correo configurado por defecto para iniciar un nuevo correo con la dirección indicada en el enlace como destinatario:

`<a href="mailto:info@ejemplo.com">Contacta con nosotros</a>`

- **Enlaces de teléfono:** Son especialmente útiles en el diseño para móviles, ya que inician la aplicación de teléfono directamente:

`<a href="tel:+34600000000">Llámanos ahora</a>`

### 3.7.4. IMÁGENES

Nuestros sitios web incluyen imágenes por diferentes motivos, como incluir logos, fotografías, ilustraciones, diagramas, etc. Darán vida y color a nuestras páginas, por lo que una buena selección de imágenes influirá definitivamente al éxito de nuestra página a través de su impacto visual. Una imagen vale más que mil palabras, y es por esto que es importante tomarse un tiempo para seleccionarlas correctamente.

Conforme un sitio web crece es importante mantenerlo ordenado, por lo que será importante mantener las imágenes en una carpeta independiente (frecuentemente llamada `images`). Incluso, en el caso de tener muchas, podemos categorizarlas creando subcarpetas temáticas dentro de la carpeta padre `images`.

Etiqueta `<img>`

Sirve para **insertar** una imagen en la página. Es un elemento en línea. No tiene etiqueta de cierre (elemento vacío).

Algunos de sus atributos permitidos son:

- **src:** indica la URL en la que está el archivo con la imagen. Como siempre, la referencia a la imagen puede ser relativa o absoluta.
- **alt:** texto con una **breve descripción** que se visualizará en pantalla en el caso de no poder mostrarse la imagen. Por temas de accesibilidad es **OBLIGATORIO** incluir este atributo. Es interpretada por software de lectura de pantalla (usados por personas con discapacidad visual) por lo que la descripción deberá ser **precisa** y **completa**. No escribamos cosas como "Imagen", sino como "Imagen de paisaje otoñal con lago en primer plano". Solo en el caso de que la imagen sea **meramente decorativa**, y no aporte ninguna información ni significado al contenido de la página, dejaremos **vacío** el contenido del atributo.
- **title:** Es el encargado de ofrecer una descripción emergente de la imagen.
- **height, width:** altura y ancho de la imagen. Si se especifican, el navegador debe modificar las medidas originales de la foto a las indicadas en estos dos atributos (en píxeles, pero únicamente el número, **sin unidades**). Es de los pocos atributos HTML relacionados con presentación que se pueden utilizar.

Su función principal es informar al navegador de las dimensiones intrínsecas (reales) de la imagen **antes de que se descargue**, evitando un problema llamado **Cumulative Layout Shift (CLS)**. El CLS es ese molesto "salto" que ves en el contenido de una página mientras se está cargando.

Con **height** y **width** el navegador, incluso antes de tener la imagen, ya sabe que mide, por ejemplo, "800x400" píxeles. Inmediatamente **reserva ese espacio** en la página. El texto se coloca correctamente a su alrededor desde el principio. Cuando la imagen termina de cargar, simplemente aparece en el hueco que ya estaba reservado para ella. **No hay saltos**, lo cual influye definitivamente de forma positiva en la experiencia de usuario.

```

```

Es muy común utilizar una imagen como área en la que hacer click para seguir un enlace, para lo que sustituiremos el texto por la imagen deseada en el contenido del elemento `<a>`:

```
<a href="https://www.youtube.com/" target="_blank"></a>
```

A la hora de introducir una imagen en nuestro contenido su posición afectará a cómo es mostrada. Veamos tres ejemplos:

```
<body>
  
  <p>Existen alrededor de 10.000 especies de aves vivas
    que habitan diferentes ecosistemas, desde el Ártico
    hasta la Antártida. Muchas especies realizan migraciones
    anuales de larga distancia, y muchas más realizan viajes
    irregulares más cortos.</p>
  <hr />
  <p>
    Existen alrededor de 10.000 especies de aves vivas
    que habitan diferentes ecosistemas, desde el Ártico
    hasta la Antártida. Muchas especies realizan migraciones
    anuales de larga distancia, y muchas más realizan viajes
    irregulares más cortos.</p>
  <hr />
  <p>Existen alrededor de 10.000 especies de aves vivas
    que habitan diferentes ecosistemas, desde el Ártico
    hasta la Antártida. Muchas especies realizan migraciones
    anuales de larga distancia, y muchas más realizan viajes
    irregulares más cortos.</p>
</body>
```



Existen alrededor de 10.000 especies de aves vivas que habitan diferentes ecosistemas, desde el Ártico hasta la Antártida. Muchas especies realizan migraciones anuales de larga distancia, y muchas más realizan viajes irregulares más cortos.

---



Existen alrededor de 10.000 especies de aves vivas que habitan diferentes ecosistemas, desde el Ártico hasta la Antártida. Muchas especies realizan migraciones anuales de larga distancia, y muchas más realizan viajes irregulares más cortos.

---

Existen alrededor de 10.000 especies de aves vivas que habitan diferentes ecosistemas,



desde el Ártico hasta la Antártida. Muchas especies realizan migraciones anuales de larga distancia, y muchas más realizan viajes irregulares más cortos.

La etiqueta <img> es de las consideradas *en línea*, por lo el siguiente contenido no aparece en una nueva línea. Un párrafo <p> será un elemento *de bloque* por lo que siempre comienza y finaliza con un salto de línea (ocupa todo el ancho de visualización del navegador).

#### 3.7.4.1. FORMATOS DE IMAGEN

La mayoría de los navegadores aceptan varios formatos de imágenes, entre ellos podemos destacar:

| JPG                                                                | GIF                                                                    | PNG                                                                                       |
|--------------------------------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Adecuado para fotografías e imágenes con tonos y detalles sutiles. | Adecuado para gráficos simples, dibujos, iconos y animaciones simples. | Adecuado para gráficos, imágenes con áreas transparentes, imágenes con detalles precisos. |
| Compresión con pérdida                                             | Compresión sin pérdida                                                 | Compresión sin pérdida                                                                    |
| 16M de colores                                                     | 256 colores                                                            | 16M de colores                                                                            |
| No permite transparencia                                           | Permite transparencia                                                  | Permite transparencia                                                                     |
| No permite animación                                               | Permite animación                                                      | No permite animación                                                                      |

Existen otros formatos, las imágenes vectoriales (SVG Scalable Vector Graphics), las cuales son independientes de la resolución. Pueden ser creadas con programas como Adobe Illustrator.

- **JPEG/JPG:** Ideal para **fotografías** e imágenes con muchos colores y degradados. Permite una buena compresión, aunque con pérdida de calidad.
- **GIF:** Usado principalmente para **animaciones cortas y simples**. Tiene una paleta de colores limitada (256 colores), por lo que no es ideal para fotografías. Cuantos más frames tenga la animación mayor será el tamaño de la imagen resultante.
- **PNG:** Perfecto para imágenes que necesitan **transparencia** (como logos sin fondo) y para gráficos con colores planos (iconos, diagramas). No tiene pérdida de calidad, pero los archivos pueden ser más pesados que los JPEG.
- **SVG:** Formato de **gráficos vectoriales**. Al no estar basado en píxeles, es infinitamente escalable sin perder calidad. Perfecto para logos, iconos e ilustraciones complejas.
- **WebP:** Un formato moderno desarrollado por Google que ofrece una compresión superior a JPEG y PNG (archivos más ligeros con la misma calidad) y soporta tanto transparencia como animaciones. Es la opción recomendada actualmente, aunque siempre hay que tener en cuenta la compatibilidad con navegadores más antiguos.

Selecciona el formato más adecuado a tus intenciones, rediménsionala al tamaño correcto para que no aparezca distorsionada y utiliza la resolución correcta. La mayoría de los navegadores trabajan con una resolución de 72 ppp (píxeles por pulgada). Imágenes de más resolución implican mayor necesidad de almacenamiento y tiempos de carga de la página sin un beneficio real.

#### 3.7.4.2. ETIQUETAS <figure> Y <figcaption>

Son etiquetas introducidas en la versión HTML5 de HTML. Las imágenes, usualmente, incluyen subtítulos o leyendas. HTML5 introdujo el elemento **<figure>** para contener la imagen junto a su pie de imagen, de modo que permanezcan asociados.

El elemento **<figcaption>**, hijo del elemento anterior, incluirá el subtítulo o pie de foto de la imagen asociada. Si utilizamos un navegador muy antiguo es posible que estas etiquetas no sean reconocidas, en cuyo caso simplemente son ignoradas.

```
<figure>
  
  <figcaption>Fig.1 - Pequeño pájaro en libertad.</figcaption>
</figure>
```

El uso de `<figure>` no se restringe a imágenes, sino que la etiqueta especifica contenido autónomo o autocontenido, por lo que puede usarse en ilustraciones, diagramas, fotos, listas de códigos, vídeos, etc.

### 3.7.5. TABLAS

Las **tablas** en HTML son utilizadas para presentar información en formato de “grid” o rejilla, de forma similar a como lo hace una hoja de cálculo tradicional. Facilitan el entendimiento de los datos presentados.

La tabla se divide en **filas** (horizontales) y **columnas** (verticales). La intersección de una fila y una columna formará una **celda** de la tabla.

Veamos las etiquetas básicas:

**<table>**: Es el elemento usado para crear la tabla. El contenido de la tabla se escribe fila a fila.

**<tr>**: Abreviatura de “table **row**” (fila de tabla), esta etiqueta se utiliza para definir filas dentro de la tabla. Cada fila contiene celdas que representan datos.

**<td>**: Abreviatura de “table **data**” (dato de tabla), esta etiqueta se utiliza para definir celdas de datos individuales dentro de una fila. Aquí es donde se coloca el contenido de la tabla, como texto, números, imágenes, etc.

**<th>**: Abreviatura de “table **header**” (encabezado de tabla), esta etiqueta se utiliza para definir celdas de encabezado dentro de una fila. Los encabezados son similares en su uso al elemento `<td>`, pero suelen contener información importante sobre las columnas o filas de la tabla y suelen estar en negrita y centradas.

Se puede utilizar el atributo **scope** en el elemento `<th>` para indicar si la cabecera corresponde a una fila o una columna. Toma los valores `row` para indicar si corresponde a una fila y `col` para indicar que es la cabecera de una columna.

```
<table>
  <caption>Notas del Trimestre</caption>
  <thead>
    <tr>
```



```

<th scope="col"></th>

<th scope="col">Matemáticas</th>
<th scope="col">Lengua</th>
<th scope="col">Ciencias</th>
</tr>
</thead>
<tbody>
<tr>
<th scope="row">Ana</th>
<td>9</td>
<td>8</td>
<td>10</td>
</tr>
<tr>
<th scope="row">Luis</th>
<td>7</td>
<td>9</td>
<td>8</td>
</tr>
</tbody>
</table>

```

### 3.7.5.1. AGRUPANDO CELDAS

#### AGRUPANDO COLUMNAS

En algunas ocasiones es interesante expandir la información en más de una columna. El atributo **colspan** se utiliza en elementos `<th>` o `<td>` para indicar cuántas celdas hay que agrupar.

```

<body>
  <table>
    <tr>
      <td colspan="2"> a </td>
    </tr>
    <tr>
      <td>b</td>
      <td>c</td>
    </tr>
  </table>
</body>

```

```
<body>
  <table>
    <thead>
      <caption>Notas del Trimestre</caption>
      <!-- row: <tr> -->
      <!-- header: <th> -->
      <tr>
        <th scope="col"></th>
        <th scope="col">Matemáticas</th>
        <th scope="col">Lengua</th>
        <th scope="col">Ciencias</th>
      </tr>
    </thead>
    <tbody>
      <!-- row: <tr> -->
      <!-- header: <th> -->
      <!-- data: <td> -->
      <tr>
        <th scope="row">Ana</th>
        <td>9</td>
        <td>8</td>
        <td>10</td>
      </tr>
      <tr>
        <th scope="row">Luis</th>
        <td>7</td>
        <td>9</td>
        <td>8</td>
      </tr>
    </tbody>
  </table>
</body>
```

# AGRUPANDO CELDAS

|          |   |
|----------|---|
| dato 1_1 |   |
| b        | c |

---

# AGRUPANDO FILAS

|   |   |
|---|---|
| a | b |
|   | c |

## AGRUPANDO FILAS

En el caso de que necesites expandir la información más allá de una fila puedes utilizar el atributo **rowspan** que se puede utilizar en elementos `<th>` o `<td>`.

```
<body>
  <table>
    <tr>
      <td rowspan="2"> a </td>
      <td>b</td>
    </tr>
    <tr>
      <td>c</td>
    </tr>
  </table>
</body>
```

### 3.7.5.2. ESTRUCTURANDO LA TABLA

Hay elementos que ayudan a **distinguir** entre el **contenido principal** de la tabla y la **primera y la última fila** (que pueden contener contenido diferente). Estos elementos ayudan a quienes usan lectores de pantalla y también permiten aplicar estilos a estas secciones de forma diferente al resto de la tabla (como verás al aprender CSS).

- **<thead>**: se utiliza para agrupar filas que contienen encabezados de tabla. A menudo se coloca en la parte superior de la tabla y se usa para definir los encabezados de la tabla. Normalmente solo hay una fila de encabezado de tabla.
- **<tfoot>**: se utiliza para agrupar filas que contienen información de pie de tabla, como totales, resúmenes o notas finales. A menudo se coloca en la parte inferior de la tabla. Normalmente solo hay una fila de pie de tabla.
- **<tbody>**: se utiliza para agrupar las filas de datos en el cuerpo principal de la tabla. Aunque no es estrictamente necesario, se utiliza para organizar mejor el contenido de la tabla, especialmente si tienes una tabla larga.
- **<caption>**: se utiliza para agregar un título o resumen a la tabla. El texto que coloques dentro de esta etiqueta aparecerá encima o debajo de la tabla, dependiendo de cómo lo estilies. De aparecer siempre tiene que ser el **primer hijo directo** de `<table>`.

```
<body>
  <table>
    <caption>Tabla con datos variados</caption>
    <thead>
      <tr>
        <th>Encabezado 1</th>
```

```

        <th>Encabezado 2</th>
    </tr>
</thead>
<tbody>
    <tr>
        <td>Dato 1</td>
        <td>Dato 2</td>
    </tr>
    <tr>
        <td>Dato 3</td>
        <td>Dato 4</td>
    </tr>
    <!-- Más filas de datos... -->
</tbody>
<tfoot>
    <tr>
        <td>Total 1</td>
        <td>Total 2</td>
    </tr>
</tfoot>
</table>
</body>

```

Atributos de <table> **OBSOLETOS:**

summary, width, frame, rules, border, cellpadding, cellspacing.

Atributos de <td> y <th> **OBSOLETOS:**

align, valign, bgcolor, width, height.

### 3.7.6. FORMULARIOS

Los formularios nos permiten, dentro de una página web, **solicitar información** al visitante para que después sea procesada por algún agente (p.ej., un servidor web, un servidor de correo, etc.). Junto con los enlaces son el método fundamental con el que podemos **interactuar** con la persona que está navegando en nuestro sitio.

En el formulario podremos solicitar **diferentes datos**, cada uno de los cuales quedará asociado a una variable. Los usuarios interactúan con los formularios a través de los llamados **campos o controles**. El **nombre** de un control viene dado por su atributo **name**. Sin este atributo el contenido del campo no viajará al servidor, por lo que no podrá ser

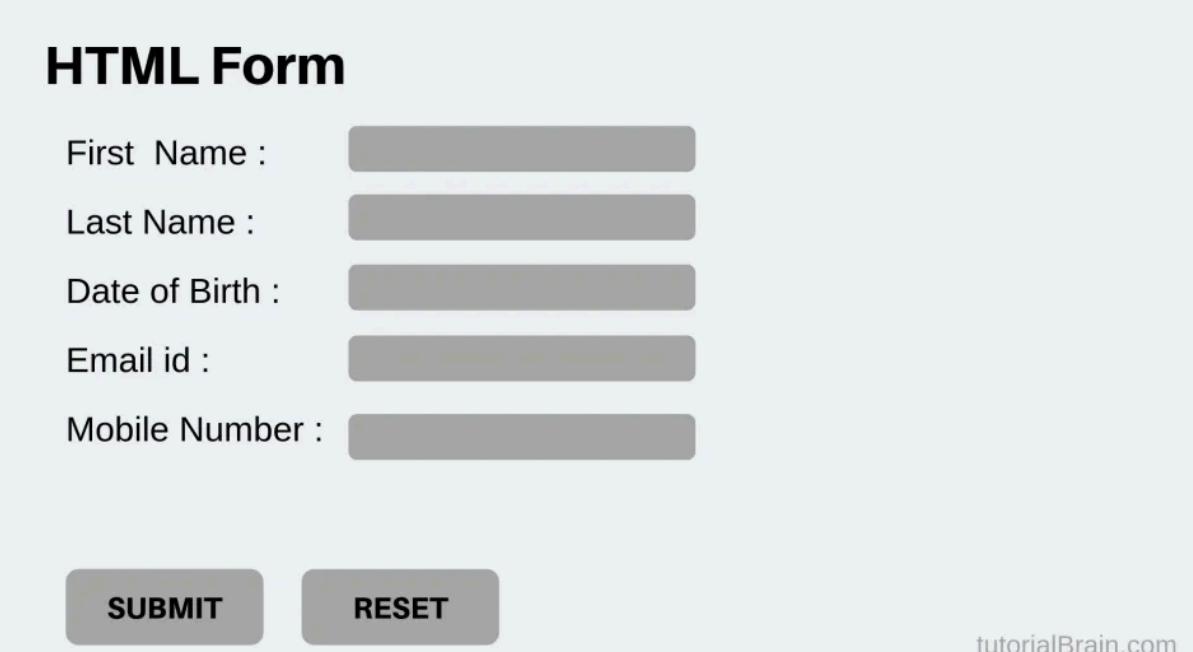
Tabla con datos variados

| Encabezado 1 | Encabezado 2 |
|--------------|--------------|
| Dato 1       | Dato 2       |
| Dato 3       | Dato 4       |
| Total 1      | Total 2      |

tratado. Es por tanto un atributo **fundamental y obligatorio** para todo elemento contenido en el formulario.

Una vez se hayan introducido los valores en los campos, el contenido de estos será enviado a la dirección (URL) donde tengamos el programa que pueda procesar las variables.

### ¿Cómo funciona un formulario?



The image shows a web form titled "HTML Form". It contains five input fields with labels: "First Name :", "Last Name :", "Date of Birth :", "Email id :", and "Mobile Number :". Below the input fields are two buttons labeled "SUBMIT" and "RESET". The form is set against a light blue background. In the bottom right corner, the text "tutorialBrain.com" is visible.

1. Un usuario completa un formulario y luego presiona un botón para enviar la información al servidor.
2. El nombre de cada control de formulario se envía al servidor junto con el valor que el usuario introduce o selecciona.
3. El servidor procesa la información utilizando un lenguaje de programación como PHP, C#, VB.NET o Java. También puede almacenar la información en una base de datos.
4. El servidor crea una nueva página para enviarla al navegador basándose en la información recibida.

La declaración del formulario se pone entre las etiquetas **<form>** y **</form>**. En el interior de la declaración se indican los elementos de entrada.

Atributos de la etiqueta **<form>** :

- **action**. Indica la dirección URI del programa al que hay que enviar los datos, es decir, del programa que va a "tratar" las variables que se envíen con el formulario.

- **method.** Permite los valores “post” y “get”. Indica el método en el que se transferirán las variables al programa.
- **enctype.** Especifica cómo se deben codificar los datos del formulario al enviarlos al servidor. Solo se usa cuando el valor del atributo method es “post”. Cuando se utiliza “get” se envían en la URL y el atributo enctype se ignora.

Sus posibles valores son:

- **application/x-www-form-urlencoded:** es el valor por defecto. Todos los caracteres se codifican antes del envío (los espacios se convierten en símbolos “+” y los caracteres especiales se convierten en valores ASCII HEX)
- **multipart/form-data:** usar si se van a adjuntar y enviar ficheros.
- **id:** Se usa para identificar el formulario de forma distinta a otros elementos de la página (y suele ser utilizado por scripts, como los que comprueban si se ha ingresado información en campos que requieren valores).
- **autocomplete.** Permite los valores “on” y “off”. El valor por defecto es on. Cuando es configurado como off, los valores <input> pertenecientes a este formulario tendrán la función de autocompletar desactivada, sin mostrar las entradas previas como posibles valores. Puede ser implementado en el elemento <form> o en cualquier elemento <input> independiente.
- **novalidate.** Una de las características de los formularios en HTML5 es la capacidad propia de validación. Los formularios son automáticamente validados (mediante elementos del tipo required, maxlength, etc.). Para evitar este comportamiento podemos usar el atributo novalidate. Para lograr lo mismo para elementos <input> específicos, existe otro atributo llamado formnovalidate. Ambos atributos son booleanos, ningún valor tiene que ser especificado para este atributo, su presencia es suficiente para activar su función.
- **onsubmit.** Sirve para ejecutar alguna acción cuando el formulario es enviado, por ejemplo alguna validación de la página en JavaScript.

El siguiente ejemplo muestra un formulario que va a ser procesado por el programa “recogida.php” cuando sea enviado. El formulario será enviado al programa usando el método HTTP “post”.

```
<form action="http://algunsitio.com/recogida.php" method="post">
  <!-- ...contenido del formulario... -->
</form>
```

Si el destino del formulario tiene el protocolo mailto los datos del formulario serán enviados por email:

```
<form action="mailto: correo@gmail.com" method="post">
  <!-- ...contenidos del formulario... -->
</form>
```



Si se utiliza el método GET se puede visualizar el valor del contenido de los campos en la url cuando hagamos submit.

```
<form action="verrespuestas.html" method="get">
```

### 3.7.6.1 MÉTODO DE ENVÍO DEL FORMULARIO

El atributo **method** del elemento **form** especifica el método HTTP usado para enviar el formulario al agente procesador. Este atributo puede tener dos valores:

- **get:** con el método HTTP "get", el conjunto de datos del formulario se agrega al URL especificado por el atributo **action** (con un signo de interrogación "?" como separador) y este nuevo URL se envía al agente procesador.
- **post:** con el método HTTP "post", el conjunto de datos del formulario se incluye en el cuerpo del formulario y se envía al agente procesador.

El método "get" debería usarse cuando el formulario no tiene efectos secundarios, esto sucede cuando no se producen cambios en el servidor después de que se envía. Por ejemplo, las búsquedas o consultas en bases de datos no tienen efectos secundarios y constituyen aplicaciones ideales del método "get".

Si el servicio asociado con el procesamiento de un formulario causa efectos secundarios (por ejemplo, si el formulario modifica una base de datos o la suscripción a un servicio), debería usarse el método "post".

Además hay que tener en cuenta que con el método "get" se realiza una única conexión con el servidor y los datos que se envíen se pueden visualizar en el navegador, mientras que con el método "post" se realizan dos conexiones distintas: una para enviar el formulario con la petición y otra para enviar los datos, de forma que estos últimos no se muestran al usuario. Pensemos por ejemplo en un formulario con usuario y contraseña; aunque protejamos la visualización en pantalla de la contraseña, con el método "get" esta se mostraría en la url que aparece en el navegador, mientras que con el método "post" no se podría ver ningún dato.

### 3.7.6.2 <input>. CAMPOS DE ENTRADA DE DATOS

El elemento **<input>** se utiliza para crear diferentes controles dentro del formulario. Es un elemento vacío, por lo que no requiere de una etiqueta de cierre. Algunos de sus atributos son:

- **name:** Cuando el servidor recibe los datos del formulario necesita saber desde qué control se envió cada uno de los datos recibidos. Éste es el atributo utilizado para esta tarea, por lo que es **fundamental incluirlo**. Cualquier control que no lo tenga no será enviado al procesar el formulario.

- **id: Identificador único** que podremos utilizar para introducir controles adicionales desde JavaScript. Es obligatorio que **todos los atributos id de una página web sean distintos**.
- **placeholder:** Especialmente en campos de tipo search, pero también en entradas de texto normales, el atributo placeholder representa una sugerencia corta, una palabra o frase provista para ayudar al usuario a ingresar la información correcta. El valor de este atributo es presentado en pantalla por los navegadores dentro del campo, como una marca de agua que desaparece cuando el elemento toma el foco.
- **required:** Este atributo booleano no dejará que el formulario sea enviado si el campo se encuentra vacío. Por ejemplo, cuando usamos el tipo email para recibir una dirección de email, el navegador comprueba si la entrada es un email válido o no, pero validará la entrada si el campo está vacío. Cuando el atributo required es incluido, la entrada será válida sólo si se cumplen las dos condiciones: que el campo no esté vacío y que el valor ingresado esté de acuerdo con los requisitos del tipo de campo.
- **autofocus:** Esta es una función que muchos desarrolladores aplicaban anteriormente utilizando el método focus() de JavaScript. Autofocus permite especificar el elemento de un formulario que debe tener foco para ingresar información cuando se carga la página, a menos que el usuario lo cambie.
- **type:** Indica el tipo de control. Puede tomar diferentes valores:
  - **type="text":** El control consiste en un campo de entrada de texto de una sola línea. Atributos adicionales:
    - **maxlength y minlength:** Se utiliza para el número máximo y mínimo de caracteres que pueden ser introducidos.
    - **size:** Sirve para indicar el tamaño "visible" del campo. En diseños modernos debería de controlarse el tamaño mediante CSS.
    - **value:** Valor inicial del campo o por defecto del campo.

```
<form action="http://www.example.com/login.php">
  <p>Username:
    <input type="text" name="username" size="15" maxlength="30"
  />
  </p>
</form>
```

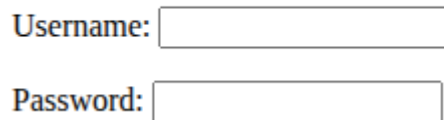
- **type="password":** Crea una caja de texto igual que al tomar el valor text pero los caracteres tecleados no son visibles para que no puedan ser vistos por otras personas. Tiene los mismos atributos adicionales que text.

```
<form action="http://www.example.com/login.php">
  <p>Username:
    <input type="text" name="username" size="15" maxlength="30"
  />
  </p>
```

```

    <p>Password:
      <input type="password" name="password" size="15"
maxlength="30" />
    </p>
  </form>

```



- **type="email"**: Indica que el campo será una email. El texto insertado será controlado por el navegador y validado como un email. Si la validación falla, el formulario no será enviado. Algunos teléfonos inteligentes también optimizan su teclado para mostrar las teclas que probablemente necesitarás al ingresar una dirección de correo electrónico (como el símbolo @). Tiene los mismos atributos adicionales que text.
- **type="search"**: El tipo search (búsqueda) no controla la entrada, es solo una indicación para los navegadores. Al detectar este tipo de campo algunos navegadores puede que cambien el diseño del elemento para ofrecer al usuario un indicio de su propósito. Tiene los mismos atributos adicionales que text.

```

<input type="search" name="busqueda" id="bus">

```

- **type="url"**: Este tipo de campo trabaja exactamente igual que el tipo email pero es específico para direcciones web. Está destinado a recibir solo URL absolutas y retornará un error si el valor es inválido. Algunos teléfonos inteligentes también optimizan su teclado para mostrar las teclas que probablemente necesitarás al ingresar una URL. Tiene los mismos atributos adicionales que text.

```

<input type="url" name="url" id="dir">

```

- **type="tel"**: Este tipo de campo es para números telefónicos. A diferencia de los tipos email y url, el tipo tel no requiere ninguna sintaxis en particular. Es solo una indicación para el navegador en caso de que necesite hacer ajustes de acuerdo al dispositivo en el que la aplicación es ejecutada. Tiene los mismos atributos adicionales que text.
- **type="number"**: Como su nombre lo indica, el tipo number es sólo válido cuando recibe una entrada numérica. Incorpora dos botones para incrementar o decrementar el valor del campo. El valor es almacenado en el atributo value. Existen algunos atributos nuevos que pueden ser útiles para este campo:

- **min:** El valor de este atributo determina el **mínimo** valor aceptado para el campo.
- **max:** El valor de este atributo determina el **máximo** valor aceptado para el campo.
- **step:** El valor de este atributo determina el tamaño en el que el valor será **incrementado o disminuido en cada paso**. Por ejemplo, si declara un valor de 5 para step en un campo que tiene un valor mínimo de 0 y máximo de 10, el navegador no le permitirá especificar valores entre 0 y 5 o entre 5 y 10.
- Todos son opcionales, y el valor por defecto para step es 1.

```
<input type="number" name="numero" id="numero" min="1" max="20" step="3">
```

- **type="range":** Este control le permite al usuario seleccionar un valor a partir de una serie de valores o rango. Normalmente es mostrado en pantalla como un puntero deslizable o un campo con flechas para seleccionar un valor entre los predeterminados, pero no existe un diseño estándar hasta el momento. Su valor es almacenado en el atributo value. Comparte atributos adicionales con el valor number. Si se desea visualizar el valor del campo por pantalla habría que utilizar JavaScript.

```
<input type="range" name="rango" id="rango1" step="3">
```

- **type="date":** Este control fue incluido para ofrecer una mejor forma de ingresar una fecha. Algunos navegadores muestran en pantalla un calendario que aparece cada vez que el usuario hace clic sobre el campo. El calendario le permite al usuario seleccionar un día que será ingresado en el campo como valor de la fecha. Gracias al tipo date ahora es el navegador el que se encarga de construir las herramientas necesarias para facilitar el ingreso de este tipo de datos. La interface no fue declarada en la especificación, por lo que cada navegador provee una propia. Atributos adicionales:
  - **value:** Fecha por defecto seleccionada.
  - **min:** Fecha mínima válida.
  - **max:** Fecha máxima válida.

```
<input type="date" name="fecha" id="entrada">
```

```
<input type="date" name="fecha" id="salida" value="2023-01-01" min="2023-01-01" max="2023-01-31">
```

- **type="week"**: Este tipo de campo ofrece una interface similar a date, pero solo para seleccionar una semana completa. Normalmente el valor esperado tiene la sintaxis 2011-W50 donde 2011 es al año y 50 es el número de la semana. Tiene los mismos atributos adicionales que date .

```
<input type="week" name="semana" id="semana1">
```

```
<input id="week" type="week" name="week" id="semana" value="2023-W10" min="2023-W01" max="2024-W52">
```

- **type="month"**: Similar a "week", pero éste es específico para seleccionar meses. Normalmente el valor esperado tiene la sintaxis "yyyy-mm". Tiene los mismos atributos adicionales que date .

```
<input id="month" type="month" name="mes">
```

```
<input id="month" type="month" name="mes" value="2018-05"
min="2018-03" max="2013-12">
```

- **type="time"**: Este campo es similar a "date", pero solo para la hora. Puede tomar el formato de hh:mm y también hh:mm:ss. Tiene los mismos atributos adicionales que date .

```
<input id="time" type="time" name="hora">
```

```
<input id="time" type="time" name="time" value="21:50:30"
min="21:45:10" max="22:00:30">
```

- **type="datetime-local"**: El tipo de campo datetime-local es para ingresar fecha y hora completa. Tiene los mismos atributos adicionales que date .

```
<input type="datetime-local" name="fechahora">
```

10/10/2022 13:49

octubre de 2022

↑

↓

L M X J V S D

26 27 28 29 30 1 2

3 4 5 6 7 8 9

10 11 12 13 14 15 16

17 18 19 20 21 22 23

24 25 26 27 28 29 30

31 1 2 3 4 5 6

Borrar

Hoy

13 48

14 49

15 50

16 51

17 52

18 53

19 54

```
<input type="datetime-local" id="meeting-time" name="meeting-time"
value="2023-01-12T19:30"
min="2023-01-07T00:00" max="2023-02-14T00:00">
```

- **type="color"**: Además de los tipos de campo para fecha y hora existe otro tipo que provee una interface predefinida similar para seleccionar colores. Atributo adicional:
  - **value**: Sirve para asignar un valor **opcional por defecto**.

```
<input type="color" name="color" id="color">
```

```
<input type="color" name="color" value="#ff0000">
```

- **type="checkbox"**: Permite definir una casilla de verificación. La casilla puede estar marcada o no. El campo se elegirá marcando una casilla. Se permite marcar varias casillas (son independientes). Las casillas que no se marcan, no se envían. Los valores de las casillas serán indicados por:
  - **value="valor"**: Valor enviado si la casilla ha sido marcada. Si no se indica tiene un valor por defecto igual a **on**.
  - **checked**: Lo añadimos para indicar que la casilla debe aparecer marcada por defecto.

```
<form action="http://www.example.com/profile.php" method="get">
  <p>Please select your favorite music service(s):
    <br>
    <input type="checkbox" name="service" value="itunes" checked>
iTunes
    <!-- Más adelante veremos una forma más correcta de poner
etiquetas a estos elementos -->
    <input type="checkbox" name="service" value="lastfm"> Last.fm
    <input type="checkbox" name="service" value="spotify">
Spotify
  </p>
</form>
```

Please select your favorite music service(s):

☒ iTunes ☐ Last.fm ☐ Spotify

- En el ejemplo anterior, al ser opciones relacionadas, todos los checkboxes tienen el **mismo nombre pero con diferente value**. No es obligatorio pero es una forma conveniente de tratar esta casuística. La otra alternativa es utilizar nombre diferentes con el mismo value, pero la primera opción es recomendable si existe relación entre las casillas de verificación. Esto permite que lenguajes de servidor (como PHP) reciban las múltiples selecciones en un único array, facilitando enormemente su procesamiento.
- **type="radio"**: Permite definir una casilla de elección. El campo se elegirá marcando una casilla. Sólo se permite marcar una de las casillas dentro del mismo grupo, por lo que se utilizará con opciones excluyentes. Los valores de las casillas serán indicados por:
  - **value="valor"**: Este es el **valor** que se **envía** si esta opción es la seleccionada.
  - **checked**: Indica si la casilla aparecerá marcada **por defecto**.
  - **IMPORTANTE**: Para que varios radio button pertenezcan al mismo grupo y pueda seleccionarse solo uno de ellos, **deben tener el mismo name con diferente value**. Es lo mismo que comentamos en el caso del checkbox, pero allí era opcional y aquí es **obligatorio**.



```
<form action="http://www.example.com/profile.php">
  <p>Please select your favorite genre:
    <br>
    <input type="radio" name="genre" value="rock" checked> Rock
    <input type="radio" name="genre" value="pop">
    Pop
    <input type="radio" name="genre" value="jazz">
    Jazz
  </p>
</form>
```

Please select your favorite genre:

☒ Rock ☐ Pop ☐ Jazz

- **type="hidden"**: El usuario no puede modificar su valor, ya que el campo no es visible. Se manda siempre con el valor indicado en el atributo value.
  - **value="valor"**: Valor que se envía para el elemento al enviar el formulario.
  - Los campos ocultos sirven para que podamos recibir información de la página, y así conocer un valor determinado pero sin mostrarlo por pantalla. También puede servir para enviar información al servidor sin que el usuario lo vea y cualquier otra cosa que el diseñador necesite. Aquí no les veremos la utilidad, pero **son fundamentales para el desarrollo de aplicaciones web**.
- **type="submit"**: Representa un botón de envío. Al pulsar este botón la información de todos los campos se envía al programa indicado en <form> (URL indicada en el atributo action de la etiqueta form).
  - **value="texto"**: Indica el texto que aparecerá en el botón.
  - **disabled**: Deshabilita el botón y no se puede pulsar sobre él. Este atributo se gestiona normalmente desde JavaScript y suele estar relacionado con controles que tiene que superar el formulario antes de ser enviado.

```
<form action="http://www.example.com/subscribe.php">
  <p>Subscribe to our email list:</p>
  <input type="text" name="email">
  <input type="submit" name="subscribe" value="Subscribe">
</form>
```

Subscribe to our email list:

En desarrollo...

- **type="reset"**: Representa un botón de reinicio. Al pulsar este botón se borra el contenido de todos los campos del formulario y se recuperan sus valores por defecto.
  - **value="texto"**: Indica el texto que aparecerá en el botón.
  - **disabled**: Deshabilita el botón y no se puede pulsar sobre él.
- **type="button"**: Permite definir un botón genérico al que se puede asociar una acción utilizando algún lenguaje de script en el cliente.
  - **value="texto"**: Indica el texto que aparecerá en el botón.
  - **disabled**: Deshabilita el botón y no se puede pulsar sobre él.
  - **onclick**: Atributo que define la función (o código) JavaScript que se ejecutará al pulsar el botón.

```
<input type="button" value="Pulsa aquí" onclick="alert('Has pulsado');">
```

**Nota:** en este ejemplo vemos que en una línea HTML en la que se aniden las comillas, estas deben alternarse, de forma que usemos las dobles y las simples para que se distinga perfectamente dónde empieza y termina la cadena de caracteres.

Un ejemplo algo más “complejo”:

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <title>Ejemplos con enlaces</title>
  <script>
    // Esto es JavaScript. No te preocupes por este código ahora
    // (aunque puedes intentar entenderlo)
    function sumarNumeros() {
      const sumando1 = document.getElementById('sumando1').value;
      const sumando2 = document.getElementById('sumando2').value;
      const resultado = document.getElementById('resultado')
      const suma = parseInt(sumando1) + parseInt(sumando2);
      resultado.value = suma;
    }
  </script>
</head>

<body>
```

```

<!-- Dos campos tipo "number" para ambos sumandos -->
<input type="number" name="sumando1" id="sumando1" step="1">
<input type="number" name="sumando2" id="sumando2" step="1">
<!-- Botón que ejecuta la función JavaScript declarada en <head> -->
<input type="button" value="+" onclick="sumarNumeros()">
<!-- Campo deshabilitado. Manipulamos su contenido desde JavaScript -->
<input type="text" name="resultado" id="resultado" disabled>
</body>

</html>

```

- **type = "file"**: Permite definir un control para enviar ficheros al servidor junto con los datos del formulario. El fichero se enviará al servidor para ser tratado allí. Los usuarios deben especificar la ruta local del archivo como contenido del control. Para ayudar con esto, los navegadores usualmente agregan un botón que, cuando es presionado, abre un navegador de archivos que permite a los usuarios elegir el archivo fácilmente. Para que la subida de archivos sea satisfactoria, el atributo **enctype** debe ser **"multipart/form-data"** y **method="post"**. Como el resto de elementos de un formulario necesita un atributo **name** (y es conveniente asignarle un **id**). Otros atributos:
  - **accept**: Indica el formato de ficheros que se aceptan. Sirve como filtro a la hora de que los archivos que el usuario ve en su computador para poder subirlos, es decir, si solo se permite subir extensiones \*.pdf cuando pulse el botón solo verá carpetas con los ficheros \*.pdf que tenga en su computador.

```

<form action="procesar_fichero.php" method="post"
enctype="multipart/form-data">
  <label for="fich1">Fichero a enviar:</label> <input type="file"
name="fich" id="fich1"
  accept=".docx, .pdf, .zip">
  <!-- Es una buena práctica incluir los MIME types para cada
extensión
  <input type="file" accept=".docx,
application/vnd.openxmlformats-officedocument.wordprocessingml.document
,
  .pdf, application/pdf,
  .zip, application/zip">
  Esto da información adicional al navegador y maximiza la
compatibilidad →
</form>

```

**Nota:** puedes consultar los MIME types para las extensiones más comunes en este enlace:

[https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/MIME\\_types/Common\\_types](https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/MIME_types/Common_types)

- **type="image"**: Permite crear un botón submit con una imagen de fondo. Los atributos `src`, `width`, `height` y `alt` funcionan igual que cuando se usan con el elemento `<img>` que ya estudiamos anteriormente.

```
<form action="http://www.example.org/subscribe.php">
  <p>Subscribe to our email list:</p>
  <input type="email" name="email" required>
  <input type="image" src="images/subscribe.jpg" width="90"
height="29" name="subscribe">
</form>
```

### 3.7.6.3 <textarea>. CAMPOS DE ENTRADA DE TEXTO MULTILÍNEA

El elemento **<textarea>** se utiliza para crear una entrada de texto multilínea. A diferencia de otros elementos de entrada, este **no es un elemento vacío**. Por lo tanto, **debe tener una etiqueta de apertura y una de cierre**.

Cualquier texto que aparezca entre las etiquetas de apertura **<textarea>** y de cierre **</textarea>** aparecerá en el cuadro de texto al cargar la página.

Si estás creando un formulario nuevo, **es más conveniente usar CSS para controlar el ancho y la altura** de un `<textarea>`. Sin embargo puedes utilizar los atributos `cols` y `rows` dentro de este elemento.

**<textarea>**

- **cols**: Indica el ancho que debe tener el área de texto (medido en número de caracteres).
- **rows**: Indica cuántas filas debe ocupar el área de texto verticalmente.

El principal propósito de estos atributos hoy en día es actuar como un **fallback** o tamaño por defecto. La mejor práctica es **usar ambos (atributos y CSS)**. **Usa rows y cols en tu HTML** para definir un tamaño base razonable. Esto asegura que si el archivo CSS por alguna razón no se carga, el usuario seguirá viendo un área de texto funcional y con un tamaño usable, en lugar de una caja diminuta y CSS para definir el estilo final y “responsive” de tu página. El CSS siempre tendrá prioridad sobre los atributos HTML para el renderizado visual.

Si el usuario no elimina ningún texto entre estas etiquetas, este mensaje se enviará al servidor junto con lo que haya escrito. (Algunos sitios usan JavaScript para borrar esta información cuando el usuario hace clic en el área de texto). También es muy habitual utilizar el atributo `placeholder`, cuyo texto es eliminado al marcar el control. Así evitamos el problema anterior.

```
<form action="http://www.example.com/comments.php">
  <p>¿Qué piensas de este artículo?</p>
  <textarea name="comments" cols="20" rows="4" placeholder="Este
texto se borra al empezar a escribir">
    Este texto se envía al servidor si no se elimina</textarea>
</form>
```

### 3.7.6.4 <label>. ETIQUETANDO LOS CONTROLES DEL FORMULARIO

El elemento **<label>** representa una etiqueta para un elemento en una interfaz de usuario. No es un elemento vacío, por lo que tiene etiqueta de cierre.

Cada control de formulario debe tener su propio elemento `<label>`, ya que esto facilita el acceso al formulario a usuarios con discapacidad visual. Puede usar de dos maneras:

1. Envolver tanto la descripción del texto como la entrada del formulario (como se muestra en la primera línea del ejemplo).
2. Mantenerse separado del control de formulario y usar el atributo **for** para indicar a qué control de formulario pertenece la etiqueta (como se muestra con los botones de opción "radio").

```
<form action="http://www.example.com/comments.php">
  <label>Edad: <input type="number" name="age" step="1"></label>
  <p>¿Hablas inglés fluido?</p>
  <input id="english_yes" type="radio" name="english" value="y"
checked>
  <label for="english_yes">Sí</label>
  <input id="english_no" type="radio" name="english" value="n">
  <label for="english_no">No</label>
</form>
```

Atributos:

- **for**: Indica a qué control de formulario pertenece la etiqueta. Observa cómo los botones de opción usan el atributo `id`. El valor del atributo `id` identifica de forma **única** un elemento de todos los demás elementos de una página. El valor del atributo `for` coincide con el del atributo `id` del control de formulario que está etiquetando. Esta técnica, que utiliza los atributos `for` e `id` de, se puede usar en

cualquier control de formulario. Cuando se usa un elemento `<label>` con una casilla de verificación (checkbox) o un botón de opción (radio), los usuarios pueden hacer clic en el control de formulario o en la etiqueta para seleccionar. En otros elementos se dará el foco al elemento de entrada asociado. El área ampliada en la que se puede hacer clic facilita el uso del formulario. La posición de la etiqueta es muy importante. Si los usuarios no saben dónde o qué información introducir, es menos probable que utilicen el formulario correctamente. Como regla general, estos son los mejores lugares para colocar etiquetas en los controles del formulario:

- Arriba o a la izquierda:
  - Entradas de texto
  - Áreas de texto
  - Cuadros de selección
  - Subida de archivos
- A la derecha:
  - Casillas de verificación individuales
  - Botones de opción individuales

### 3.7.6.5 `<select>`. CAJAS DE LISTAS DESPLEGABLES

El elemento **`<select>`** permite a los usuarios seleccionar una o varias opciones de una lista desplegable. No es un elemento vacío, por lo que es obligatorio incluir una etiqueta de cierre. Como siempre, tendremos que incluir un atributo **`name`** (y muy frecuentemente un **`id`**). Además tenemos los atributos:

- **`multiple`**: Permite a los usuarios seleccionar más de un valor en la lista desplegable (pulsando la tecla “control” en PC o “command” en un Mac). Es conveniente avisar de que esta opción está disponible.
- **`size`**: Definimos el número de líneas visibles en la lista a la vez. Si por ejemplo la lista tiene 7 elementos y fijamos el atributo `size="2"`, se mostrarán sólo dos elementos de la lista, debiendo movernos para ver el resto.

Una lista desplegable es una buena alternativa a una lista de “radio buttons” (sin selección múltiple) o “checkboxes” (con selección múltiple) en las que hay una lista de opciones muy grande, como podría ser una lista de países.

#### 3.7.6.5.1 `<option>` y `<optgroup>`. AÑADIENDO OPCIONES A LA LISTA

Las diferentes opciones de la lista `<select>` se indican con el elemento hijo **`<option>`**, cuya etiqueta cierre es `</option>`. Las palabras entre ambas etiquetas se mostrarán en la lista desplegable como una opción. Atributos:

- **`value="valor"`**: Valor a enviar al servidor si se selecciona el elemento de la lista junto con el nombre del elemento. Si no se incluye se devuelve el texto de la opción.

- **selected:** Indicamos que esta opción aparece seleccionada por defecto. Si no se incluye se muestra la primera opción por defecto, y esta opción será enviada si no se selecciona nada.

Por otro lado, el elemento **<optgroup>** permite agrupar de una manera lógica las opciones de un elemento **<select>**. Esto es particularmente útil cuando el usuario debe elegir entre una larga lista de opciones. Es más fácil apreciar y recordar grupos de opciones relacionadas que una larga lista de opciones sueltas. Entre la etiqueta inicial **<optgroup>** y la final **</optgroup>** se incluyen las opciones del grupo. El **atributo label** del elemento **optgroup** especifica el rótulo del grupo de opciones.

```
<form action="http://www.example.com/comments.php">
  <select name="pais" size="8" multiple>
    <optgroup label="Europa"> <!--Primer grupo de opciones:
Europa-->
      <option value="es" selected>España</option>
      <option value="fr">Francia</option>
      <option value="dt">Alemania</option>
    </optgroup> <!-- Cierre del grupo "Europa"-->
    <optgroup label="Asia"><!--Segundo grupo de opciones:
Asia-->
      <option value="jp">Japón</option>
      <option value="ch">China</option>
      <option value="in">India</option>
    </optgroup> <!--Cierre del grupo "Asia"-->
  </select>
</form>
```



### 3.7.6.6 <datalist>. LISTA DE SUGERENCIAS

La etiqueta `<datalist>` crea una **lista de sugerencias de autocompletado** para un campo `<input>`. Es un híbrido entre un campo de texto normal y un menú desplegable `<select>`.

El usuario puede:

- Escribir libremente en el campo de texto.
- O elegir una de las opciones predefinidas que aparecen a medida que escribe.

Piensa en ello como la barra de búsqueda de Google: te ofrece sugerencias, pero no estás obligado a elegir ninguna.

El elemento `<datalist>` es un elemento específico de formularios usado para construir una lista de ítems que luego, mediante el atributo **list**, será asociada y usada como sugerencias en un campo del formulario. Cada una de las posibles sugerencias se define mediante un elemento **<option>** de la misma forma que lo ya visto para un elemento `<select>`

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo de Datalist</title>
</head>
<body>

  <label for="navegador">Elige tu navegador favorito:</label>

  <input type="text" id="navegador" name="navegador"
list="lista-navegadores">

  <datalist id="lista-navegadores">
    <option value="Chrome"></option>
    <option value="Firefox"></option>
    <option value="Safari"></option>
    <option value="Edge"></option>
    <option value="Opera"></option>
    <option value="Brave"></option>
  </datalist>

</body>
</html>
```



### 3.7.6.7 <fieldset> Y <legend>. ESTRUCTURANDO EL FORMULARIO

El elemento **<fieldset>** (grupo de campos) permite agrupar temáticamente controles y rótulos relacionados.

Gracias al agrupamiento de controles es más fácil para los usuarios entender su propósito y al mismo tiempo se facilita la navegación con agentes de usuario visuales y la navegación por voz para agentes de usuario basados en voz.

Casi todos los navegadores mostrarán una línea alrededor de los elementos agrupados, cuya apariencia puede ajustarse utilizando CSS.

Por su lado el elemento **<legend>**, situado justo detrás de la apertura de <fieldset>, permite asignar un título al grupo.

```
<form action="recogida.php" method="post">
  <fieldset>
    <legend>Información de contacto:</legend>
    <p>
      <label for="name">Nombre:</label><br>
      <input type="text" id="name" name="name">
    </p>
    <p>
      <label for="ape">Apellidos:</label><br>
      <input type="text" id="ape" name="ape">
    </p>
    <p>
      <label for="email">Email:</label><br>
      <input type="email" id="email" name="email">
    </p>
  </fieldset>
</form>
```

### 3.7.6.8 ALGUNOS ATRIBUTOS COMUNES A ELEMENTOS DE <form>

- **tabindex**: Especifica la posición del elemento actual dentro del orden de tabulación del documento. El valor debe estar entre 0 y 32767. Los siguientes elementos soportan el atributo `tabindex`: `a`, `area`, `button`, `input`, `object`, `select` y `textarea`.
- **placeholder**: Utilizado en elementos de entrada de texto, el atributo `placeholder` representa una palabra o sugerencia corta para ayudar al usuario a ingresar la

información correcta. El valor de este atributo es presentado en pantalla por los navegadores dentro del campo, como una marca de agua que desaparece cuando comenzamos a escribir.

- **required:** Este atributo no dejará que el formulario sea enviado si el campo se encuentra vacío. Por ejemplo, cuando usamos el tipo email para recibir una dirección de email, el navegador comprueba si la entrada es un email válido o no, pero validará la entrada si el campo está vacío. Cuando el atributo required es incluido, la entrada será válida sólo si se cumplen las dos condiciones: que el campo no esté vacío y que el valor ingresado esté de acuerdo con los requisitos del tipo de campo.
- **autofocus:** permite especificar el elemento de un formulario que debe tener foco para ingresar información cuando se carga la página.
- **pattern:** El atributo pattern es para propósitos de validación. Usa expresiones regulares para personalizar reglas de validación, como por ejemplo validar que un código postal conste de 5 números. El atributo pattern nos permite crear nuestro propio tipo de campo para controlar esta clase de valores admitidos. El atributo title es fundamental cuando usas pattern, ya que el texto que pongas en él se mostrará al usuario como una ayuda o un mensaje de error si el patrón no se cumple.

[https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Regular\\_Expressions/Cheatsheet](https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Regular_Expressions/Cheatsheet)

Aquí tienes un resumen de los patrones más utilizados, con ejemplos listos para usar.

### ¿Qué significan los símbolos básicos?

- [a-z] — Cualquier letra minúscula.
- [A-Z] — Cualquier letra mayúscula.
- [0-9] — Cualquier número.
- {n} — Se repite n veces exactamente.
- {n,m} — Se repite entre n y m veces.
- + — Se repite una o más veces.
- \* — Se repite cero o más veces.
- ^ — Inicio de la cadena.
- \$ — Fin de la cadena.

### Ejemplos

- Usuario (solo letras y números, de 3 a 15 caracteres)

Permite sólo letras (mayúsculas y minúsculas) y números. Sin espacios ni símbolos.

```
<label for="usuario">Usuario:</label>
  <input type="text" id="usuario" name="usuario"
pattern="[A-Za-z0-9]{3,15}"
  title="Solo letras y números, entre 3 y 15 caracteres.">
```

- Código Postal (5 dígitos exactos)

Fuerza a que el valor introducido sean exactamente 5 números

```
<label for="cp">Código Postal:</label>
    <input type="text" id="cp" name="cp" pattern="[0-9]{5}"
title="El código postal debe tener 5 dígitos.">
```

- DNI Español (8 números y 1 letra)

Valida el formato estándar de un DNI en España.

```
<label for="dni">DNI:</label>
    <input type="text" id="dni" name="dni"
pattern="[0-9]{8}[A-Za-z]{1}"
    title="Formato de DNI no válido (8 números y 1 letra).">
```

- Teléfono Fijo o Móvil Español (9 dígitos)

Valida que se introduzcan 9 números.

```
<label for="telefono">Teléfono:</label>
    <input type="tel" id="telefono" name="telefono"
pattern="[0-9]{9}"
    title="El número de teléfono debe tener 9 dígitos.">
```

- **form**: El atributo form es una adición útil que nos permite declarar elementos para un formulario fuera del ámbito de las etiquetas `<form>`. Hasta ahora, para construir un formulario teníamos que escribir las etiquetas `<form>` de apertura y cierre y luego declarar cada elemento del formulario entre ellas. A partir de HTML5 podemos insertar los elementos en cualquier parte del código y luego hacer referencia al formulario que pertenecen usando su nombre y el atributo form:

```
<body>
    <form id="envio-nombre" action="enviar_datos.php" method="post">
        <label for="name">Nombre:</label>
        <input type="text" id="name" name="name"
form="envio-nombre">
    </form>
    <label for="ape">Apellidos:</label>
    <input type="text" id="ape" name="apellidos"
form="envio-nombre">
    <form id="envio-mail" action="mailto:form2@form2.com">
        <label for="email">Email:</label>
```

```
        <input type="email" id="email" name="email"
form="envio-mail">
    </form>
    <label for="tf">Teléfono:</label>
    <input type="tel" id="tf" name="telefono" form="envio-mail">
    <input type="submit" value="Envía Form 1" form="envio-nombre">
    <input type="submit" value="Envía Form 2" form="envio-mail">
</body>
```

Para finalizar, se debe tener en cuenta que en un documento HTML pueden existir varios formularios. Cada formulario puede tener una acción diferente, lo que significa que puede enviar los datos de cada formulario a diferentes páginas o servidores y utilizar diferentes métodos de envío (como GET o POST) para enviar los datos.